

Artificial Intelligence

A Modern Approach

Third Edition



**PRENTICE HALL SERIES
IN ARTIFICIAL INTELLIGENCE**
Stuart Russell and Peter Norvig, Editors

FORSYTH & PONCE
GRAHAM
JURAFSKY & MARTIN
NEAPOLITAN
RUSSELL & NORVIG

Computer Vision: A Modern Approach
ANSI Common Lisp
Speech and Language Processing, 2nd ed.
Learning Bayesian Networks
Artificial Intelligence: A Modern Approach, 3rd ed.

Artificial Intelligence

A Modern Approach

Third Edition

Stuart J. Russell and Peter Norvig

Contributing writers:

Ernest Davis
Douglas D. Edwards
David Forsyth
Nicholas J. Hay
Jitendra M. Malik
Vibhu Mittal
Mehran Sahami
Sebastian Thrun

Prentice Hall

Upper Saddle River Boston Columbus San Francisco New York
Indianapolis London Toronto Sydney Singapore Tokyo Montreal
Dubai Madrid Hong Kong Mexico City Munich Paris Amsterdam Cape Town

Vice President and Editorial Director, ECS: Marcia J. Horton
Editor-in-Chief: Michael Hirsch
Executive Editor: Tracy Dunkelberger
Assistant Editor: Melinda Haggerty
Editorial Assistant: Allison Michael
Vice President, Production: Vince O'Brien
Senior Managing Editor: Scott Disanno
Production Editor: Jane Bonnell
Senior Operations Supervisor: Alan Fischer
Operations Specialist: Lisa McDowell
Marketing Manager: Erin Davis
Marketing Assistant: Mack Patterson
Cover Designers: Kirsten Sims and Geoffrey Cassar
Cover Images: Stan Honda/Getty, Library of Congress, NASA, National Museum of Rome,
Peter Norvig, Ian Parker, Shutterstock, Time Life/Getty
Interior Designers: Stuart Russell and Peter Norvig
Copy Editor: Mary Lou Nohr
Art Editor: Greg Dulles
Media Editor: Daniel Sandin
Media Project Manager: Danielle Leone

**Copyright © 2010, 2003, 1995 by Pearson Education, Inc.,
Upper Saddle River, New Jersey 07458.**

All rights reserved. Manufactured in the United States of America. This publication is protected by Copyright and permissions should be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. To obtain permission(s) to use materials from this work, please submit a written request to Pearson Higher Education, Permissions Department, 1 Lake Street, Upper Saddle River, NJ 07458.

The author and publisher of this book have used their best efforts in preparing this book. These efforts include the development, research, and testing of the theories and programs to determine their effectiveness. The author and publisher make no warranty of any kind, expressed or implied, with regard to these programs or the documentation contained in this book. The author and publisher shall not be liable in any event for incidental or consequential damages in connection with, or arising out of, the furnishing, performance, or use of these programs.

Library of Congress Cataloging-in-Publication Data on File

Prentice Hall
is an imprint of



www.pearsonhighered.com

10 9 8 7 6 5 4 3 2 1
ISBN-13: 978-0-13-604259-4
ISBN-10: 0-13-604259-7

For Loy, Gordon, Lucy, George, and Isaac — S.J.R.

For Kris, Isabella, and Juliet — P.N.

This page intentionally left blank

Preface

Artificial Intelligence (AI) is a big field, and this is a big book. We have tried to explore the full breadth of the field, which encompasses logic, probability, and continuous mathematics; perception, reasoning, learning, and action; and everything from microelectronic devices to robotic planetary explorers. The book is also big because we go into some depth.

The subtitle of this book is “A Modern Approach.” The intended meaning of this rather empty phrase is that we have tried to synthesize what is now known into a common framework, rather than trying to explain each subfield of AI in its own historical context. We apologize to those whose subfields are, as a result, less recognizable.

New to this edition

This edition captures the changes in AI that have taken place since the last edition in 2003. There have been important applications of AI technology, such as the widespread deployment of practical speech recognition, machine translation, autonomous vehicles, and household robotics. There have been algorithmic landmarks, such as the solution of the game of checkers. And there has been a great deal of theoretical progress, particularly in areas such as probabilistic reasoning, machine learning, and computer vision. Most important from our point of view is the continued evolution in how we think about the field, and thus how we organize the book. The major changes are as follows:

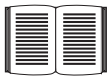
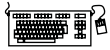
- We place more emphasis on partially observable and nondeterministic environments, especially in the nonprobabilistic settings of search and planning. The concepts of *belief state* (a set of possible worlds) and *state estimation* (maintaining the belief state) are introduced in these settings; later in the book, we add probabilities.
- In addition to discussing the types of environments and types of agents, we now cover in more depth the types of *representations* that an agent can use. We distinguish among *atomic* representations (in which each state of the world is treated as a black box), *factored* representations (in which a state is a set of attribute/value pairs), and *structured* representations (in which the world consists of objects and relations between them).
- Our coverage of planning goes into more depth on contingent planning in partially observable environments and includes a new approach to hierarchical planning.
- We have added new material on first-order probabilistic models, including *open-universe* models for cases where there is uncertainty as to what objects exist.
- We have completely rewritten the introductory machine-learning chapter, stressing a wider variety of more modern learning algorithms and placing them on a firmer theoretical footing.
- We have expanded coverage of Web search and information extraction, and of techniques for learning from very large data sets.
- 20% of the citations in this edition are to works published after 2003.
- We estimate that about 20% of the material is brand new. The remaining 80% reflects older work but has been largely rewritten to present a more unified picture of the field.

Overview of the book

The main unifying theme is the idea of an **intelligent agent**. We define AI as the study of agents that receive percepts from the environment and perform actions. Each such agent implements a function that maps percept sequences to actions, and we cover different ways to represent these functions, such as reactive agents, real-time planners, and decision-theoretic systems. We explain the role of learning as extending the reach of the designer into unknown environments, and we show how that role constrains agent design, favoring explicit knowledge representation and reasoning. We treat robotics and vision not as independently defined problems, but as occurring in the service of achieving goals. We stress the importance of the task environment in determining the appropriate agent design.

Our primary aim is to convey the *ideas* that have emerged over the past fifty years of AI research and the past two millennia of related work. We have tried to avoid excessive formality in the presentation of these ideas while retaining precision. We have included pseudocode algorithms to make the key ideas concrete; our pseudocode is described in Appendix B.

This book is primarily intended for use in an undergraduate course or course sequence. The book has 27 chapters, each requiring about a week's worth of lectures, so working through the whole book requires a two-semester sequence. A one-semester course can use selected chapters to suit the interests of the instructor and students. The book can also be used in a graduate-level course (perhaps with the addition of some of the primary sources suggested in the bibliographical notes). Sample syllabi are available at the book's Web site, aima.cs.berkeley.edu. The only prerequisite is familiarity with basic concepts of computer science (algorithms, data structures, complexity) at a sophomore level. Freshman calculus and linear algebra are useful for some of the topics; the required mathematical background is supplied in Appendix A.



NEW TERM

Exercises are given at the end of each chapter. Exercises requiring significant programming are marked with a **keyboard** icon. These exercises can best be solved by taking advantage of the code repository at aima.cs.berkeley.edu. Some of them are large enough to be considered term projects. A number of exercises require some investigation of the literature; these are marked with a **book** icon.

Throughout the book, important points are marked with a *pointing* icon. We have included an extensive index of around 6,000 items to make it easy to find things in the book. Wherever a **new term** is first defined, it is also marked in the margin.

About the Web site

aima.cs.berkeley.edu, the Web site for the book, contains

- implementations of the algorithms in the book in several programming languages,
- a list of over 1000 schools that have used the book, many with links to online course materials and syllabi,
- an annotated list of over 800 links to sites around the Web with useful AI content,
- a chapter-by-chapter list of supplementary material and links,
- instructions on how to join a discussion group for the book,

- instructions on how to contact the authors with questions or comments,
- instructions on how to report errors in the book, in the likely event that some exist, and
- slides and other materials for instructors.

About the cover

The cover depicts the final position from the decisive game 6 of the 1997 match between chess champion Garry Kasparov and program DEEP BLUE. Kasparov, playing Black, was forced to resign, making this the first time a computer had beaten a world champion in a chess match. Kasparov is shown at the top. To his left is the Asimo humanoid robot and to his right is Thomas Bayes (1702–1761), whose ideas about probability as a measure of belief underlie much of modern AI technology. Below that we see a Mars Exploration Rover, a robot that landed on Mars in 2004 and has been exploring the planet ever since. To the right is Alan Turing (1912–1954), whose fundamental work defined the fields of computer science in general and artificial intelligence in particular. At the bottom is Shakey (1966–1972), the first robot to combine perception, world-modeling, planning, and learning. With Shakey is project leader Charles Rosen (1917–2002). At the bottom right is Aristotle (384 B.C.–322 B.C.), who pioneered the study of logic; his work was state of the art until the 19th century (copy of a bust by Lysippos). At the bottom left, lightly screened behind the authors' names, is a planning algorithm by Aristotle from *De Motu Animalium* in the original Greek. Behind the title is a portion of the CPSC Bayesian network for medical diagnosis (Pradhan *et al.*, 1994). Behind the chess board is part of a Bayesian logic model for detecting nuclear explosions from seismic signals.

Credits: Stan Honda/Getty (Kasparaov), Library of Congress (Bayes), NASA (Mars rover), National Museum of Rome (Aristotle), Peter Norvig (book), Ian Parker (Berkeley skyline), Shutterstock (Asimo, Chess pieces), Time Life/Getty (Shakey, Turing).

Acknowledgments

This book would not have been possible without the many contributors whose names did not make it to the cover. Jitendra Malik and David Forsyth wrote Chapter 24 (computer vision) and Sebastian Thrun wrote Chapter 25 (robotics). Vibhu Mittal wrote part of Chapter 22 (natural language). Nick Hay, Mehran Sahami, and Ernest Davis wrote some of the exercises. Zoran Duric (George Mason), Thomas C. Henderson (Utah), Leon Reznik (RIT), Michael Gourley (Central Oklahoma) and Ernest Davis (NYU) reviewed the manuscript and made helpful suggestions. We thank Ernie Davis in particular for his tireless ability to read multiple drafts and help improve the book. Nick Hay whipped the bibliography into shape and on deadline stayed up to 5:30 AM writing code to make the book better. Jon Barron formatted and improved the diagrams in this edition, while Tim Huang, Mark Paskin, and Cynthia Bruyns helped with diagrams and algorithms in previous editions. Ravi Mohan and Ciaran O'Reilly wrote and maintain the Java code examples on the Web site. John Canny wrote the robotics chapter for the first edition and Douglas Edwards researched the historical notes. Tracy Dunkelberger, Allison Michael, Scott Disanno, and Jane Bonnell at Pearson tried their best to keep us on schedule and made many helpful suggestions. Most helpful of all has

been Julie Sussman, P.P.A., who read every chapter and provided extensive improvements. In previous editions we had proofreaders who would tell us when we left out a comma and said *which* when we meant *that*; Julie told us when we left out a minus sign and said x_i when we meant x_j . For every typo or confusing explanation that remains in the book, rest assured that Julie has fixed at least five. She persevered even when a power failure forced her to work by lantern light rather than LCD glow.

Stuart would like to thank his parents for their support and encouragement and his wife, Loy Sheflott, for her endless patience and boundless wisdom. He hopes that Gordon, Lucy, George, and Isaac will soon be reading this book after they have forgiven him for working so long on it. RUGS (Russell's Unusual Group of Students) have been unusually helpful, as always.

Peter would like to thank his parents (Torsten and Gerda) for getting him started, and his wife (Kris), children (Bella and Juliet), colleagues, and friends for encouraging and tolerating him through the long hours of writing and longer hours of rewriting.

We both thank the librarians at Berkeley, Stanford, and NASA and the developers of CiteSeer, Wikipedia, and Google, who have revolutionized the way we do research. We can't acknowledge all the people who have used the book and made suggestions, but we would like to note the especially helpful comments of Gagan Aggarwal, Eyal Amir, Ion Androutsopoulos, Krzysztof Apt, Warren Haley Armstrong, Ellery Aziel, Jeff Van Baalen, Darius Bacon, Brian Baker, Shumeet Baluja, Don Barker, Tony Barrett, James Newton Bass, Don Beal, Howard Beck, Wolfgang Bibel, John Binder, Larry Bookman, David R. Boxall, Ronen Brafman, John Bresina, Gerhard Brewka, Selmer Bringsjord, Carla Brodley, Chris Brown, Emma Brunskill, Wilhelm Burger, Lauren Burka, Carlos Bustamante, Joao Cachopo, Murray Campbell, Norman Carver, Emmanuel Castro, Anil Chakravarthy, Dan Chisarick, Berthe Choueiry, Roberto Cipolla, David Cohen, James Coleman, Julie Ann Comparini, Corinna Cortes, Gary Cottrell, Ernest Davis, Tom Dean, Rina Dechter, Tom Dietterich, Peter Drake, Chuck Dyer, Doug Edwards, Robert Egginton, Asma'a El-Budrawy, Barbara Engelhardt, Kutluhan Erol, Oren Etzioni, Hana Filip, Douglas Fisher, Jeffrey Forbes, Ken Ford, Eric Fosler-Lussier, John Fosler, Jeremy Frank, Alex Franz, Bob Futrelle, Marek Galecki, Stefan Gerberding, Stuart Gill, Sabine Glesner, Seth Golub, Gosta Grahne, Russ Greiner, Eric Grimson, Barbara Grosz, Larry Hall, Steve Hanks, Othar Hansson, Ernst Heinz, Jim Hendler, Christoph Herrmann, Paul Hilfinger, Robert Holte, Vasant Honavar, Tim Huang, Seth Hutchinson, Joost Jacob, Mark Jelasity, Magnus Johansson, Istvan Jonyer, Dan Jurafsky, Leslie Kaelbling, Keiji Kanazawa, Surekha Kasibhatla, Simon Kasif, Henry Kautz, Gernot Kerschbaumer, Max Khesin, Richard Kirby, Dan Klein, Kevin Knight, Roland Koenig, Sven Koenig, Daphne Koller, Rich Korf, Benjamin Kuipers, James Kurien, John Lafferty, John Laird, Gus Larsen, John Lazzaro, Jon LeBlanc, Jason Leatherman, Frank Lee, Jon Lehto, Edward Lim, Phil Long, Pierre Louveaux, Don Loveland, Sridhar Mahadevan, Tony Mancill, Jim Martin, Andy Mayer, John McCarthy, David McGrane, Jay Mendelsohn, Risto Miikkulanien, Brian Milch, Steve Minton, Vibhu Mittal, Mehryar Mohri, Leora Morgenstern, Stephen Muggleton, Kevin Murphy, Ron Musick, Sung Myaeng, Eric Nadeau, Lee Naish, Pandu Nayak, Bernhard Nebel, Stuart Nelson, XuanLong Nguyen, Nils Nilsson, Illah Nourbakhsh, Ali Nouri, Arthur Nunes-Harwitt, Steve Omohundro, David Page, David Palmer, David Parkes, Ron Parr, Mark

Paskin, Tony Passera, Amit Patel, Michael Pazzani, Fernando Pereira, Joseph Perla, Wim Pijls, Ira Pohl, Martha Pollack, David Poole, Bruce Porter, Malcolm Pradhan, Bill Pringle, Lorraine Prior, Greg Provan, William Rapaport, Deepak Ravichandran, Ioannis Refanidis, Philip Resnik, Francesca Rossi, Sam Roweis, Richard Russell, Jonathan Schaeffer, Richard Scherl, Hinrich Schuetze, Lars Schuster, Bart Selman, Soheil Shams, Stuart Shapiro, Jude Shavlik, Yoram Singer, Satinder Singh, Daniel Sleator, David Smith, Bryan So, Robert Sroull, Lynn Stein, Larry Stephens, Andreas Stolcke, Paul Stradling, Devika Subramanian, Marek Suchenek, Rich Sutton, Jonathan Tash, Austin Tate, Bas Terwijn, Olivier Teytaud, Michael Thielscher, William Thompson, Sebastian Thrun, Eric Tiedemann, Mark Torrance, Randall Upham, Paul Utgoff, Peter van Beek, Hal Varian, Paulina Varshavskaya, Sunil Vemuri, Vandiver Verma, Ubbo Visser, Jim Waldo, Toby Walsh, Bonnie Webber, Dan Weld, Michael Wellman, Kamin Whitehouse, Michael Dean White, Brian Williams, David Wolfe, Jason Wolfe, Bill Woods, Alden Wright, Jay Yagnik, Mark Yasuda, Richard Yen, Eliezer Yudkowsky, Weixiong Zhang, Ming Zhao, Shlomo Zilberstein, and our esteemed colleague Anonymous Reviewer.

About the Authors

Stuart Russell was born in 1962 in Portsmouth, England. He received his B.A. with first-class honours in physics from Oxford University in 1982, and his Ph.D. in computer science from Stanford in 1986. He then joined the faculty of the University of California at Berkeley, where he is a professor of computer science, director of the Center for Intelligent Systems, and holder of the Smith–Zadeh Chair in Engineering. In 1990, he received the Presidential Young Investigator Award of the National Science Foundation, and in 1995 he was cowinner of the Computers and Thought Award. He was a 1996 Miller Professor of the University of California and was appointed to a Chancellor’s Professorship in 2000. In 1998, he gave the Forsythe Memorial Lectures at Stanford University. He is a Fellow and former Executive Council member of the American Association for Artificial Intelligence. He has published over 100 papers on a wide range of topics in artificial intelligence. His other books include *The Use of Knowledge in Analogy and Induction* and (with Eric Wefald) *Do the Right Thing: Studies in Limited Rationality*.

Peter Norvig is currently Director of Research at Google, Inc., and was the director responsible for the core Web search algorithms from 2002 to 2005. He is a Fellow of the American Association for Artificial Intelligence and the Association for Computing Machinery. Previously, he was head of the Computational Sciences Division at NASA Ames Research Center, where he oversaw NASA’s research and development in artificial intelligence and robotics, and chief scientist at Junglee, where he helped develop one of the first Internet information extraction services. He received a B.S. in applied mathematics from Brown University and a Ph.D. in computer science from the University of California at Berkeley. He received the Distinguished Alumni and Engineering Innovation awards from Berkeley and the Exceptional Achievement Medal from NASA. He has been a professor at the University of Southern California and a research faculty member at Berkeley. His other books are *Paradigms of AI Programming: Case Studies in Common Lisp* and *Verbmobil: A Translation System for Face-to-Face Dialog* and *Intelligent Help Systems for UNIX*.

Contents

I Artificial Intelligence

1	Introduction	1
1.1	What Is AI?	1
1.2	The Foundations of Artificial Intelligence	5
1.3	The History of Artificial Intelligence	16
1.4	The State of the Art	28
1.5	Summary, Bibliographical and Historical Notes, Exercises	29
2	Intelligent Agents	34
2.1	Agents and Environments	34
2.2	Good Behavior: The Concept of Rationality	36
2.3	The Nature of Environments	40
2.4	The Structure of Agents	46
2.5	Summary, Bibliographical and Historical Notes, Exercises	59

II Problem-solving

3	Solving Problems by Searching	64
3.1	Problem-Solving Agents	64
3.2	Example Problems	69
3.3	Searching for Solutions	75
3.4	Uninformed Search Strategies	81
3.5	Informed (Heuristic) Search Strategies	92
3.6	Heuristic Functions	102
3.7	Summary, Bibliographical and Historical Notes, Exercises	108
4	Beyond Classical Search	120
4.1	Local Search Algorithms and Optimization Problems	120
4.2	Local Search in Continuous Spaces	129
4.3	Searching with Nondeterministic Actions	133
4.4	Searching with Partial Observations	138
4.5	Online Search Agents and Unknown Environments	147
4.6	Summary, Bibliographical and Historical Notes, Exercises	153
5	Adversarial Search	161
5.1	Games	161
5.2	Optimal Decisions in Games	163
5.3	Alpha–Beta Pruning	167
5.4	Imperfect Real-Time Decisions	171
5.5	Stochastic Games	177

5.6	Partially Observable Games	180
5.7	State-of-the-Art Game Programs	185
5.8	Alternative Approaches	187
5.9	Summary, Bibliographical and Historical Notes, Exercises	189
6	Constraint Satisfaction Problems	202
6.1	Defining Constraint Satisfaction Problems	202
6.2	Constraint Propagation: Inference in CSPs	208
6.3	Backtracking Search for CSPs	214
6.4	Local Search for CSPs	220
6.5	The Structure of Problems	222
6.6	Summary, Bibliographical and Historical Notes, Exercises	227
III	Knowledge, reasoning, and planning	
7	Logical Agents	234
7.1	Knowledge-Based Agents	235
7.2	The Wumpus World	236
7.3	Logic	240
7.4	Propositional Logic: A Very Simple Logic	243
7.5	Propositional Theorem Proving	249
7.6	Effective Propositional Model Checking	259
7.7	Agents Based on Propositional Logic	265
7.8	Summary, Bibliographical and Historical Notes, Exercises	274
8	First-Order Logic	285
8.1	Representation Revisited	285
8.2	Syntax and Semantics of First-Order Logic	290
8.3	Using First-Order Logic	300
8.4	Knowledge Engineering in First-Order Logic	307
8.5	Summary, Bibliographical and Historical Notes, Exercises	313
9	Inference in First-Order Logic	322
9.1	Propositional vs. First-Order Inference	322
9.2	Unification and Lifting	325
9.3	Forward Chaining	330
9.4	Backward Chaining	337
9.5	Resolution	345
9.6	Summary, Bibliographical and Historical Notes, Exercises	357
10	Classical Planning	366
10.1	Definition of Classical Planning	366
10.2	Algorithms for Planning as State-Space Search	373
10.3	Planning Graphs	379

10.4	Other Classical Planning Approaches	387
10.5	Analysis of Planning Approaches	392
10.6	Summary, Bibliographical and Historical Notes, Exercises	393
11	Planning and Acting in the Real World	401
11.1	Time, Schedules, and Resources	401
11.2	Hierarchical Planning	406
11.3	Planning and Acting in Nondeterministic Domains	415
11.4	Multiagent Planning	425
11.5	Summary, Bibliographical and Historical Notes, Exercises	430
12	Knowledge Representation	437
12.1	Ontological Engineering	437
12.2	Categories and Objects	440
12.3	Events	446
12.4	Mental Events and Mental Objects	450
12.5	Reasoning Systems for Categories	453
12.6	Reasoning with Default Information	458
12.7	The Internet Shopping World	462
12.8	Summary, Bibliographical and Historical Notes, Exercises	467
IV	Uncertain knowledge and reasoning	
13	Quantifying Uncertainty	480
13.1	Acting under Uncertainty	480
13.2	Basic Probability Notation	483
13.3	Inference Using Full Joint Distributions	490
13.4	Independence	494
13.5	Bayes' Rule and Its Use	495
13.6	The Wumpus World Revisited	499
13.7	Summary, Bibliographical and Historical Notes, Exercises	503
14	Probabilistic Reasoning	510
14.1	Representing Knowledge in an Uncertain Domain	510
14.2	The Semantics of Bayesian Networks	513
14.3	Efficient Representation of Conditional Distributions	518
14.4	Exact Inference in Bayesian Networks	522
14.5	Approximate Inference in Bayesian Networks	530
14.6	Relational and First-Order Probability Models	539
14.7	Other Approaches to Uncertain Reasoning	546
14.8	Summary, Bibliographical and Historical Notes, Exercises	551
15	Probabilistic Reasoning over Time	566
15.1	Time and Uncertainty	566

15.2	Inference in Temporal Models	570
15.3	Hidden Markov Models	578
15.4	Kalman Filters	584
15.5	Dynamic Bayesian Networks	590
15.6	Keeping Track of Many Objects	599
15.7	Summary, Bibliographical and Historical Notes, Exercises	603
16	Making Simple Decisions	610
16.1	Combining Beliefs and Desires under Uncertainty	610
16.2	The Basis of Utility Theory	611
16.3	Utility Functions	615
16.4	Multiattribute Utility Functions	622
16.5	Decision Networks	626
16.6	The Value of Information	628
16.7	Decision-Theoretic Expert Systems	633
16.8	Summary, Bibliographical and Historical Notes, Exercises	636
17	Making Complex Decisions	645
17.1	Sequential Decision Problems	645
17.2	Value Iteration	652
17.3	Policy Iteration	656
17.4	Partially Observable MDPs	658
17.5	Decisions with Multiple Agents: Game Theory	666
17.6	Mechanism Design	679
17.7	Summary, Bibliographical and Historical Notes, Exercises	684
V	Learning	
18	Learning from Examples	693
18.1	Forms of Learning	693
18.2	Supervised Learning	695
18.3	Learning Decision Trees	697
18.4	Evaluating and Choosing the Best Hypothesis	708
18.5	The Theory of Learning	713
18.6	Regression and Classification with Linear Models	717
18.7	Artificial Neural Networks	727
18.8	Nonparametric Models	737
18.9	Support Vector Machines	744
18.10	Ensemble Learning	748
18.11	Practical Machine Learning	753
18.12	Summary, Bibliographical and Historical Notes, Exercises	757
19	Knowledge in Learning	768
19.1	A Logical Formulation of Learning	768

19.2	Knowledge in Learning	777
19.3	Explanation-Based Learning	780
19.4	Learning Using Relevance Information	784
19.5	Inductive Logic Programming	788
19.6	Summary, Bibliographical and Historical Notes, Exercises	797
20	Learning Probabilistic Models	802
20.1	Statistical Learning	802
20.2	Learning with Complete Data	806
20.3	Learning with Hidden Variables: The EM Algorithm	816
20.4	Summary, Bibliographical and Historical Notes, Exercises	825
21	Reinforcement Learning	830
21.1	Introduction	830
21.2	Passive Reinforcement Learning	832
21.3	Active Reinforcement Learning	839
21.4	Generalization in Reinforcement Learning	845
21.5	Policy Search	848
21.6	Applications of Reinforcement Learning	850
21.7	Summary, Bibliographical and Historical Notes, Exercises	853
VI	Communicating, perceiving, and acting	
22	Natural Language Processing	860
22.1	Language Models	860
22.2	Text Classification	865
22.3	Information Retrieval	867
22.4	Information Extraction	873
22.5	Summary, Bibliographical and Historical Notes, Exercises	882
23	Natural Language for Communication	888
23.1	Phrase Structure Grammars	888
23.2	Syntactic Analysis (Parsing)	892
23.3	Augmented Grammars and Semantic Interpretation	897
23.4	Machine Translation	907
23.5	Speech Recognition	912
23.6	Summary, Bibliographical and Historical Notes, Exercises	918
24	Perception	928
24.1	Image Formation	929
24.2	Early Image-Processing Operations	935
24.3	Object Recognition by Appearance	942
24.4	Reconstructing the 3D World	947
24.5	Object Recognition from Structural Information	957

24.6	Using Vision	961
24.7	Summary, Bibliographical and Historical Notes, Exercises	965
25	Robotics	971
25.1	Introduction	971
25.2	Robot Hardware	973
25.3	Robotic Perception	978
25.4	Planning to Move	986
25.5	Planning Uncertain Movements	993
25.6	Moving	997
25.7	Robotic Software Architectures	1003
25.8	Application Domains	1006
25.9	Summary, Bibliographical and Historical Notes, Exercises	1010
VII	Conclusions	
26	Philosophical Foundations	1020
26.1	Weak AI: Can Machines Act Intelligently?	1020
26.2	Strong AI: Can Machines Really Think?	1026
26.3	The Ethics and Risks of Developing Artificial Intelligence	1034
26.4	Summary, Bibliographical and Historical Notes, Exercises	1040
27	AI: The Present and Future	1044
27.1	Agent Components	1044
27.2	Agent Architectures	1047
27.3	Are We Going in the Right Direction?	1049
27.4	What If AI Does Succeed?	1051
A	Mathematical background	1053
A.1	Complexity Analysis and $O()$ Notation	1053
A.2	Vectors, Matrices, and Linear Algebra	1055
A.3	Probability Distributions	1057
B	Notes on Languages and Algorithms	1060
B.1	Defining Languages with Backus–Naur Form (BNF)	1060
B.2	Describing Algorithms with Pseudocode	1061
B.3	Online Help	1062
	Bibliography	1063
	Index	1095

1

INTRODUCTION

In which we try to explain why we consider artificial intelligence to be a subject most worthy of study, and in which we try to decide what exactly it is, this being a good thing to decide before embarking.

INTELLIGENCE

We call ourselves *Homo sapiens*—man the wise—because our **intelligence** is so important to us. For thousands of years, we have tried to understand *how we think*; that is, how a mere handful of matter can perceive, understand, predict, and manipulate a world far larger and more complicated than itself. The field of **artificial intelligence**, or AI, goes further still: it attempts not just to understand but also to *build* intelligent entities.

ARTIFICIAL
INTELLIGENCE

AI is one of the newest fields in science and engineering. Work started in earnest soon after World War II, and the name itself was coined in 1956. Along with molecular biology, AI is regularly cited as the “field I would most like to be in” by scientists in other disciplines. A student in physics might reasonably feel that all the good ideas have already been taken by Galileo, Newton, Einstein, and the rest. AI, on the other hand, still has openings for several full-time Einsteins and Edisons.

AI currently encompasses a huge variety of subfields, ranging from the general (learning and perception) to the specific, such as playing chess, proving mathematical theorems, writing poetry, driving a car on a crowded street, and diagnosing diseases. AI is relevant to any intellectual task; it is truly a universal field.

1.1 WHAT IS AI?

RATIONALITY

We have claimed that AI is exciting, but we have not said what it *is*. In Figure 1.1 we see eight definitions of AI, laid out along two dimensions. The definitions on top are concerned with *thought processes* and *reasoning*, whereas the ones on the bottom address *behavior*. The definitions on the left measure success in terms of fidelity to *human* performance, whereas the ones on the right measure against an *ideal* performance measure, called **rationality**. A system is rational if it does the “right thing,” given what it knows.

Historically, all four approaches to AI have been followed, each by different people with different methods. A human-centered approach must be in part an empirical science, in-

Thinking Humanly “The exciting new effort to make computers think . . . <i>machines with minds</i> , in the full and literal sense.” (Haugeland, 1985) “[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . .” (Bellman, 1978)	Thinking Rationally “The study of mental faculties through the use of computational models.” (Charniak and McDermott, 1985) “The study of the computations that make it possible to perceive, reason, and act.” (Winston, 1992)
Acting Humanly “The art of creating machines that perform functions that require intelligence when performed by people.” (Kurzweil, 1990) “The study of how to make computers do things at which, at the moment, people are better.” (Rich and Knight, 1991)	Acting Rationally “Computational Intelligence is the study of the design of intelligent agents.” (Poole <i>et al.</i> , 1998) “AI . . . is concerned with intelligent behavior in artifacts.” (Nilsson, 1998)
Figure 1.1 Some definitions of artificial intelligence, organized into four categories.	

volving observations and hypotheses about human behavior. A rationalist¹ approach involves a combination of mathematics and engineering. The various group have both disparaged and helped each other. Let us look at the four approaches in more detail.

1.1.1 Acting humanly: The Turing Test approach

TURING TEST

The **Turing Test**, proposed by Alan Turing (1950), was designed to provide a satisfactory operational definition of intelligence. A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer. Chapter 26 discusses the details of the test and whether a computer would really be intelligent if it passed. For now, we note that programming a computer to pass a rigorously applied test provides plenty to work on. The computer would need to possess the following capabilities:

NATURAL LANGUAGE
PROCESSING
KNOWLEDGE
REPRESENTATION
AUTOMATED
REASONING

- **natural language processing** to enable it to communicate successfully in English;
- **knowledge representation** to store what it knows or hears;
- **automated reasoning** to use the stored information to answer questions and to draw new conclusions;
- **machine learning** to adapt to new circumstances and to detect and extrapolate patterns.

MACHINE LEARNING

¹ By distinguishing between *human* and *rational* behavior, we are not suggesting that humans are necessarily “irrational” in the sense of “emotionally unstable” or “insane.” One merely need note that we are not perfect: not all chess players are grandmasters; and, unfortunately, not everyone gets an A on the exam. Some systematic errors in human reasoning are cataloged by Kahneman *et al.* (1982).

TOTAL TURING TEST

Turing's test deliberately avoided direct physical interaction between the interrogator and the computer, because *physical* simulation of a person is unnecessary for intelligence. However, the so-called **total Turing Test** includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to pass physical objects "through the hatch." To pass the total Turing Test, the computer will need

COMPUTER VISION

- **computer vision** to perceive objects, and

ROBOTICS

- **robotics** to manipulate objects and move about.

These six disciplines compose most of AI, and Turing deserves credit for designing a test that remains relevant 60 years later. Yet AI researchers have devoted little effort to passing the Turing Test, believing that it is more important to study the underlying principles of intelligence than to duplicate an exemplar. The quest for "artificial flight" succeeded when the Wright brothers and others stopped imitating birds and started using wind tunnels and learning about aerodynamics. Aeronautical engineering texts do not define the goal of their field as making "machines that fly so exactly like pigeons that they can fool even other pigeons."

1.1.2 Thinking humanly: The cognitive modeling approach

COGNITIVE SCIENCE

If we are going to say that a given program thinks like a human, we must have some way of determining how humans think. We need to get *inside* the actual workings of human minds. There are three ways to do this: through introspection—trying to catch our own thoughts as they go by; through psychological experiments—observing a person in action; and through brain imaging—observing the brain in action. Once we have a sufficiently precise theory of the mind, it becomes possible to express the theory as a computer program. If the program's input-output behavior matches corresponding human behavior, that is evidence that some of the program's mechanisms could also be operating in humans. For example, Allen Newell and Herbert Simon, who developed GPS, the "General Problem Solver" (Newell and Simon, 1961), were not content merely to have their program solve problems correctly. They were more concerned with comparing the trace of its reasoning steps to traces of human subjects solving the same problems. The interdisciplinary field of **cognitive science** brings together computer models from AI and experimental techniques from psychology to construct precise and testable theories of the human mind.

Cognitive science is a fascinating field in itself, worthy of several textbooks and at least one encyclopedia (Wilson and Keil, 1999). We will occasionally comment on similarities or differences between AI techniques and human cognition. Real cognitive science, however, is necessarily based on experimental investigation of actual humans or animals. We will leave that for other books, as we assume the reader has only a computer for experimentation.

In the early days of AI there was often confusion between the approaches: an author would argue that an algorithm performs well on a task and that it is *therefore* a good model of human performance, or vice versa. Modern authors separate the two kinds of claims; this distinction has allowed both AI and cognitive science to develop more rapidly. The two fields continue to fertilize each other, most notably in computer vision, which incorporates neurophysiological evidence into computational models.

1.1.3 Thinking rationally: The “laws of thought” approach

SYLLOGISM

The Greek philosopher Aristotle was one of the first to attempt to codify “right thinking,” that is, irrefutable reasoning processes. His **sylogisms** provided patterns for argument structures that always yielded correct conclusions when given correct premises—for example, “Socrates is a man; all men are mortal; therefore, Socrates is mortal.” These laws of thought were supposed to govern the operation of the mind; their study initiated the field called **logic**.

LOGIC

LOGICIST

Logicians in the 19th century developed a precise notation for statements about all kinds of objects in the world and the relations among them. (Contrast this with ordinary arithmetic notation, which provides only for statements about *numbers*.) By 1965, programs existed that could, in principle, solve *any* solvable problem described in logical notation. (Although if no solution exists, the program might loop forever.) The so-called **logicist** tradition within artificial intelligence hopes to build on such programs to create intelligent systems.

There are two main obstacles to this approach. First, it is not easy to take informal knowledge and state it in the formal terms required by logical notation, particularly when the knowledge is less than 100% certain. Second, there is a big difference between solving a problem “in principle” and solving it in practice. Even problems with just a few hundred facts can exhaust the computational resources of any computer unless it has some guidance as to which reasoning steps to try first. Although both of these obstacles apply to *any* attempt to build computational reasoning systems, they appeared first in the logicist tradition.

1.1.4 Acting rationally: The rational agent approach

AGENT

RATIONAL AGENT

An **agent** is just something that acts (*agent* comes from the Latin *agere*, to do). Of course, all computer programs do something, but computer agents are expected to do more: operate autonomously, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals. A **rational agent** is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.

In the “laws of thought” approach to AI, the emphasis was on correct inferences. Making correct inferences is sometimes *part* of being a rational agent, because one way to act rationally is to reason logically to the conclusion that a given action will achieve one’s goals and then to act on that conclusion. On the other hand, correct inference is not *all* of rationality; in some situations, there is no provably correct thing to do, but something must still be done. There are also ways of acting rationally that cannot be said to involve inference. For example, recoiling from a hot stove is a reflex action that is usually more successful than a slower action taken after careful deliberation.

All the skills needed for the Turing Test also allow an agent to act rationally. Knowledge representation and reasoning enable agents to reach good decisions. We need to be able to generate comprehensible sentences in natural language to get by in a complex society. We need learning not only for erudition, but also because it improves our ability to generate effective behavior.

The rational-agent approach has two advantages over the other approaches. First, it is more general than the “laws of thought” approach because correct inference is just one of several possible mechanisms for achieving rationality. Second, it is more amenable to



scientific development than are approaches based on human behavior or human thought. The standard of rationality is mathematically well defined and completely general, and can be “unpacked” to generate agent designs that provably achieve it. Human behavior, on the other hand, is well adapted for one specific environment and is defined by, well, the sum total of all the things that humans do. *This book therefore concentrates on general principles of rational agents and on components for constructing them.* We will see that despite the apparent simplicity with which the problem can be stated, an enormous variety of issues come up when we try to solve it. Chapter 2 outlines some of these issues in more detail.

One important point to keep in mind: We will see before too long that achieving perfect rationality—always doing the right thing—is not feasible in complicated environments. The computational demands are just too high. For most of the book, however, we will adopt the working hypothesis that perfect rationality is a good starting point for analysis. It simplifies the problem and provides the appropriate setting for most of the foundational material in the field. Chapters 5 and 17 deal explicitly with the issue of **limited rationality**—acting appropriately when there is not enough time to do all the computations one might like.

LIMITED
RATIONALITY

1.2 THE FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

In this section, we provide a brief history of the disciplines that contributed ideas, viewpoints, and techniques to AI. Like any history, this one is forced to concentrate on a small number of people, events, and ideas and to ignore others that also were important. We organize the history around a series of questions. We certainly would not wish to give the impression that these questions are the only ones the disciplines address or that the disciplines have all been working toward AI as their ultimate fruition.

1.2.1 Philosophy

- Can formal rules be used to draw valid conclusions?
- How does the mind arise from a physical brain?
- Where does knowledge come from?
- How does knowledge lead to action?

Aristotle (384–322 B.C.), whose bust appears on the front cover of this book, was the first to formulate a precise set of laws governing the rational part of the mind. He developed an informal system of syllogisms for proper reasoning, which in principle allowed one to generate conclusions mechanically, given initial premises. Much later, Ramon Lull (d. 1315) had the idea that useful reasoning could actually be carried out by a mechanical artifact. Thomas Hobbes (1588–1679) proposed that reasoning was like numerical computation, that “we add and subtract in our silent thoughts.” The automation of computation itself was already well under way. Around 1500, Leonardo da Vinci (1452–1519) designed but did not build a mechanical calculator; recent reconstructions have shown the design to be functional. The first known calculating machine was constructed around 1623 by the German scientist Wilhelm Schickard (1592–1635), although the Pascaline, built in 1642 by Blaise Pascal (1623–1662),

is more famous. Pascal wrote that “the arithmetical machine produces effects which appear nearer to thought than all the actions of animals.” Gottfried Wilhelm Leibniz (1646–1716) built a mechanical device intended to carry out operations on concepts rather than numbers, but its scope was rather limited. Leibniz did surpass Pascal by building a calculator that could add, subtract, multiply, and take roots, whereas the Pascaline could only add and subtract. Some speculated that machines might not just do calculations but actually be able to think and act on their own. In his 1651 book *Leviathan*, Thomas Hobbes suggested the idea of an “artificial animal,” arguing “For what is the heart but a spring; and the nerves, but so many strings; and the joints, but so many wheels.”

RATIONALISM

DUALISM

MATERIALISM

It’s one thing to say that the mind operates, at least in part, according to logical rules, and to build physical systems that emulate some of those rules; it’s another to say that the mind itself *is* such a physical system. René Descartes (1596–1650) gave the first clear discussion of the distinction between mind and matter and of the problems that arise. One problem with a purely physical conception of the mind is that it seems to leave little room for free will: if the mind is governed entirely by physical laws, then it has no more free will than a rock “deciding” to fall toward the center of the earth. Descartes was a strong advocate of the power of reasoning in understanding the world, a philosophy now called **rationalism**, and one that counts Aristotle and Leibniz as members. But Descartes was also a proponent of **dualism**. He held that there is a part of the human mind (or soul or spirit) that is outside of nature, exempt from physical laws. Animals, on the other hand, did not possess this dual quality; they could be treated as machines. An alternative to dualism is **materialism**, which holds that the brain’s operation according to the laws of physics *constitutes* the mind. Free will is simply the way that the perception of available choices appears to the choosing entity.

EMPIRICISM

INDUCTION

LOGICAL POSITIVISM
OBSERVATION
SENTENCES

CONFIRMATION
THEORY

Given a physical mind that manipulates knowledge, the next problem is to establish the source of knowledge. The **empiricism** movement, starting with Francis Bacon’s (1561–1626) *Novum Organum*,² is characterized by a dictum of John Locke (1632–1704): “Nothing is in the understanding, which was not first in the senses.” David Hume’s (1711–1776) *A Treatise of Human Nature* (Hume, 1739) proposed what is now known as the principle of **induction**: that general rules are acquired by exposure to repeated associations between their elements. Building on the work of Ludwig Wittgenstein (1889–1951) and Bertrand Russell (1872–1970), the famous Vienna Circle, led by Rudolf Carnap (1891–1970), developed the doctrine of **logical positivism**. This doctrine holds that all knowledge can be characterized by logical theories connected, ultimately, to **observation sentences** that correspond to sensory inputs; thus logical positivism combines rationalism and empiricism.³ The **confirmation theory** of Carnap and Carl Hempel (1905–1997) attempted to analyze the acquisition of knowledge from experience. Carnap’s book *The Logical Structure of the World* (1928) defined an explicit computational procedure for extracting knowledge from elementary experiences. It was probably the first theory of mind as a computational process.

² The *Novum Organum* is an update of Aristotle’s *Organon*, or instrument of thought. Thus Aristotle can be seen as both an empiricist and a rationalist.

³ In this picture, all meaningful statements can be verified or falsified either by experimentation or by analysis of the meaning of the words. Because this rules out most of metaphysics, as was the intention, logical positivism was unpopular in some circles.

The final element in the philosophical picture of the mind is the connection between knowledge and action. This question is vital to AI because intelligence requires action as well as reasoning. Moreover, only by understanding how actions are justified can we understand how to build an agent whose actions are justifiable (or rational). Aristotle argued (in *De Motu Animalium*) that actions are justified by a logical connection between goals and knowledge of the action's outcome (the last part of this extract also appears on the front cover of this book, in the original Greek):

But how does it happen that thinking is sometimes accompanied by action and sometimes not, sometimes by motion, and sometimes not? It looks as if almost the same thing happens as in the case of reasoning and making inferences about unchanging objects. But in that case the end is a speculative proposition . . . whereas here the conclusion which results from the two premises is an action. . . . I need covering; a cloak is a covering. I need a cloak. What I need, I have to make; I need a cloak. I have to make a cloak. And the conclusion, the "I have to make a cloak," is an action.

In the *Nicomachean Ethics* (Book III. 3, 1112b), Aristotle further elaborates on this topic, suggesting an algorithm:

We deliberate not about ends, but about means. For a doctor does not deliberate whether he shall heal, nor an orator whether he shall persuade, . . . They assume the end and consider how and by what means it is attained, and if it seems easily and best produced thereby; while if it is achieved by one means only they consider *how* it will be achieved by this and by what means *this* will be achieved, till they come to the first cause, . . . and what is last in the order of analysis seems to be first in the order of becoming. And if we come on an impossibility, we give up the search, e.g., if we need money and this cannot be got; but if a thing appears possible we try to do it.

Aristotle's algorithm was implemented 2300 years later by Newell and Simon in their GPS program. We would now call it a regression planning system (see Chapter 10).

Goal-based analysis is useful, but does not say what to do when several actions will achieve the goal or when no action will achieve it completely. Antoine Arnauld (1612–1694) correctly described a quantitative formula for deciding what action to take in cases like this (see Chapter 16). John Stuart Mill's (1806–1873) book *Utilitarianism* (Mill, 1863) promoted the idea of rational decision criteria in all spheres of human activity. The more formal theory of decisions is discussed in the following section.

1.2.2 Mathematics

- What are the formal rules to draw valid conclusions?
- What can be computed?
- How do we reason with uncertain information?

Philosophers staked out some of the fundamental ideas of AI, but the leap to a formal science required a level of mathematical formalization in three fundamental areas: logic, computation, and probability.

The idea of formal logic can be traced back to the philosophers of ancient Greece, but its mathematical development really began with the work of George Boole (1815–1864), who

worked out the details of propositional, or Boolean, logic (Boole, 1847). In 1879, Gottlob Frege (1848–1925) extended Boole’s logic to include objects and relations, creating the first-order logic that is used today.⁴ Alfred Tarski (1902–1983) introduced a theory of reference that shows how to relate the objects in a logic to objects in the real world.

ALGORITHM

The next step was to determine the limits of what could be done with logic and computation. The first nontrivial **algorithm** is thought to be Euclid’s algorithm for computing greatest common divisors. The word *algorithm* (and the idea of studying them) comes from al-Khowarazmi, a Persian mathematician of the 9th century, whose writings also introduced Arabic numerals and algebra to Europe. Boole and others discussed algorithms for logical deduction, and, by the late 19th century, efforts were under way to formalize general mathematical reasoning as logical deduction. In 1930, Kurt Gödel (1906–1978) showed that there exists an effective procedure to prove any true statement in the first-order logic of Frege and Russell, but that first-order logic could not capture the principle of mathematical induction needed to characterize the natural numbers. In 1931, Gödel showed that limits on deduction do exist. His **incompleteness theorem** showed that in any formal theory as strong as Peano arithmetic (the elementary theory of natural numbers), there are true statements that are undecidable in the sense that they have no proof within the theory.

INCOMPLETENESS
THEOREM

This fundamental result can also be interpreted as showing that some functions on the integers cannot be represented by an algorithm—that is, they cannot be computed. This motivated Alan Turing (1912–1954) to try to characterize exactly which functions *are* **computable**—capable of being computed. This notion is actually slightly problematic because the notion of a computation or effective procedure really cannot be given a formal definition. However, the Church–Turing thesis, which states that the Turing machine (Turing, 1936) is capable of computing any computable function, is generally accepted as providing a sufficient definition. Turing also showed that there were some functions that no Turing machine can compute. For example, no machine can tell *in general* whether a given program will return an answer on a given input or run forever.

COMPUTABLE

Although decidability and computability are important to an understanding of computation, the notion of **tractability** has had an even greater impact. Roughly speaking, a problem is called intractable if the time required to solve instances of the problem grows exponentially with the size of the instances. The distinction between polynomial and exponential growth in complexity was first emphasized in the mid-1960s (Cobham, 1964; Edmonds, 1965). It is important because exponential growth means that even moderately large instances cannot be solved in any reasonable time. Therefore, one should strive to divide the overall problem of generating intelligent behavior into tractable subproblems rather than intractable ones.

TRACTABILITY

NP-COMPLETENESS

How can one recognize an intractable problem? The theory of **NP-completeness**, pioneered by Steven Cook (1971) and Richard Karp (1972), provides a method. Cook and Karp showed the existence of large classes of canonical combinatorial search and reasoning problems that are NP-complete. Any problem class to which the class of NP-complete problems can be reduced is likely to be intractable. (Although it has not been proved that NP-complete

⁴ Frege’s proposed notation for first-order logic—an arcane combination of textual and geometric features—never became popular.

problems are necessarily intractable, most theoreticians believe it.) These results contrast with the optimism with which the popular press greeted the first computers—“Electronic Super-Brains” that were “Faster than Einstein!” Despite the increasing speed of computers, careful use of resources will characterize intelligent systems. Put crudely, the world is an *extremely* large problem instance! Work in AI has helped explain why some instances of NP-complete problems are hard, yet others are easy (Cheeseman *et al.*, 1991).

PROBABILITY

Besides logic and computation, the third great contribution of mathematics to AI is the theory of **probability**. The Italian Gerolamo Cardano (1501–1576) first framed the idea of probability, describing it in terms of the possible outcomes of gambling events. In 1654, Blaise Pascal (1623–1662), in a letter to Pierre Fermat (1601–1665), showed how to predict the future of an unfinished gambling game and assign average payoffs to the gamblers. Probability quickly became an invaluable part of all the quantitative sciences, helping to deal with uncertain measurements and incomplete theories. James Bernoulli (1654–1705), Pierre Laplace (1749–1827), and others advanced the theory and introduced new statistical methods. Thomas Bayes (1702–1761), who appears on the front cover of this book, proposed a rule for updating probabilities in the light of new evidence. Bayes’ rule underlies most modern approaches to uncertain reasoning in AI systems.

1.2.3 Economics

- How should we make decisions so as to maximize payoff?
- How should we do this when others may not go along?
- How should we do this when the payoff may be far in the future?

UTILITY

The science of economics got its start in 1776, when Scottish philosopher Adam Smith (1723–1790) published *An Inquiry into the Nature and Causes of the Wealth of Nations*. While the ancient Greeks and others had made contributions to economic thought, Smith was the first to treat it as a science, using the idea that economies can be thought of as consisting of individual agents maximizing their own economic well-being. Most people think of economics as being about money, but economists will say that they are really studying how people make choices that lead to preferred outcomes. When McDonald’s offers a hamburger for a dollar, they are asserting that they would prefer the dollar and hoping that customers will prefer the hamburger. The mathematical treatment of “preferred outcomes” or **utility** was first formalized by Léon Walras (pronounced “Valrasse”) (1834–1910) and was improved by Frank Ramsey (1931) and later by John von Neumann and Oskar Morgenstern in their book *The Theory of Games and Economic Behavior* (1944).

DECISION THEORY

Decision theory, which combines probability theory with utility theory, provides a formal and complete framework for decisions (economic or otherwise) made under uncertainty—that is, in cases where probabilistic descriptions appropriately capture the decision maker’s environment. This is suitable for “large” economies where each agent need pay no attention to the actions of other agents as individuals. For “small” economies, the situation is much more like a **game**: the actions of one player can significantly affect the utility of another (either positively or negatively). Von Neumann and Morgenstern’s development of **game theory** (see also Luce and Raiffa, 1957) included the surprising result that, for some games,

GAME THEORY

a rational agent should adopt policies that are (or least appear to be) randomized. Unlike decision theory, game theory does not offer an unambiguous prescription for selecting actions.

OPERATIONS
RESEARCH

For the most part, economists did not address the third question listed above, namely, how to make rational decisions when payoffs from actions are not immediate but instead result from several actions taken *in sequence*. This topic was pursued in the field of **operations research**, which emerged in World War II from efforts in Britain to optimize radar installations, and later found civilian applications in complex management decisions. The work of Richard Bellman (1957) formalized a class of sequential decision problems called **Markov decision processes**, which we study in Chapters 17 and 21.

SATISFICING

Work in economics and operations research has contributed much to our notion of rational agents, yet for many years AI research developed along entirely separate paths. One reason was the apparent complexity of making rational decisions. The pioneering AI researcher Herbert Simon (1916–2001) won the Nobel Prize in economics in 1978 for his early work showing that models based on **satisficing**—making decisions that are “good enough,” rather than laboriously calculating an optimal decision—gave a better description of actual human behavior (Simon, 1947). Since the 1990s, there has been a resurgence of interest in decision-theoretic techniques for agent systems (Wellman, 1995).

1.2.4 Neuroscience

- How do brains process information?

NEUROSCIENCE

Neuroscience is the study of the nervous system, particularly the brain. Although the exact way in which the brain enables thought is one of the great mysteries of science, the fact that it *does* enable thought has been appreciated for thousands of years because of the evidence that strong blows to the head can lead to mental incapacitation. It has also long been known that human brains are somehow different; in about 335 B.C. Aristotle wrote, “Of all the animals, man has the largest brain in proportion to his size.”⁵ Still, it was not until the middle of the 18th century that the brain was widely recognized as the seat of consciousness. Before then, candidate locations included the heart and the spleen.

NEURON

Paul Broca’s (1824–1880) study of aphasia (speech deficit) in brain-damaged patients in 1861 demonstrated the existence of localized areas of the brain responsible for specific cognitive functions. In particular, he showed that speech production was localized to the portion of the left hemisphere now called Broca’s area.⁶ By that time, it was known that the brain consisted of nerve cells, or **neurons**, but it was not until 1873 that Camillo Golgi (1843–1926) developed a staining technique allowing the observation of individual neurons in the brain (see Figure 1.2). This technique was used by Santiago Ramon y Cajal (1852–1934) in his pioneering studies of the brain’s neuronal structures.⁷ Nicolas Rashevsky (1936, 1938) was the first to apply mathematical models to the study of the nervous system.

⁵ Since then, it has been discovered that the tree shrew (*Scandentia*) has a higher ratio of brain to body mass.

⁶ Many cite Alexander Hood (1824) as a possible prior source.

⁷ Golgi persisted in his belief that the brain’s functions were carried out primarily in a continuous medium in which neurons were embedded, whereas Cajal propounded the “neuronal doctrine.” The two shared the Nobel prize in 1906 but gave mutually antagonistic acceptance speeches.

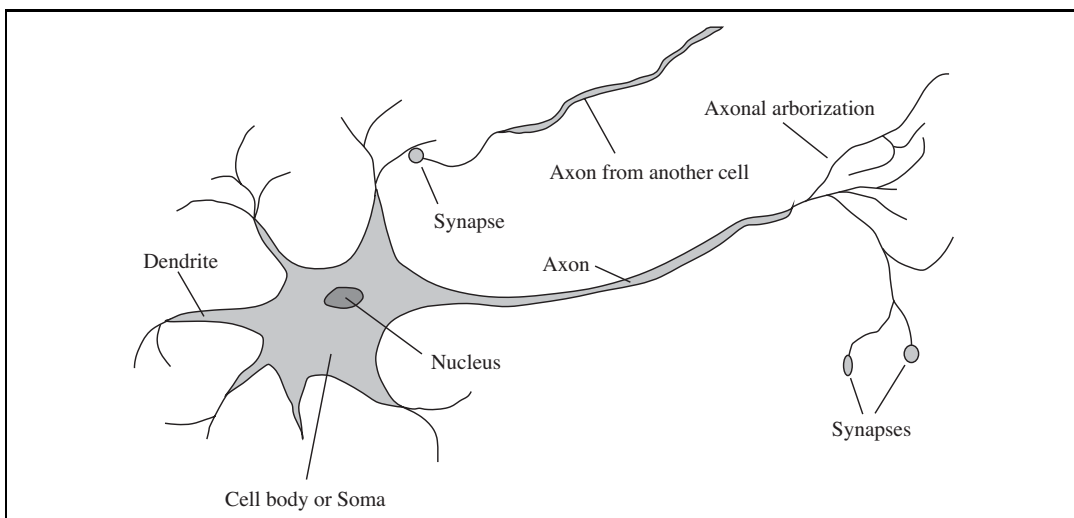


Figure 1.2 The parts of a nerve cell or neuron. Each neuron consists of a cell body, or soma, that contains a cell nucleus. Branching out from the cell body are a number of fibers called dendrites and a single long fiber called the axon. The axon stretches out for a long distance, much longer than the scale in this diagram indicates. Typically, an axon is 1 cm long (100 times the diameter of the cell body), but can reach up to 1 meter. A neuron makes connections with 10 to 100,000 other neurons at junctions called synapses. Signals are propagated from neuron to neuron by a complicated electrochemical reaction. The signals control brain activity in the short term and also enable long-term changes in the connectivity of neurons. These mechanisms are thought to form the basis for learning in the brain. Most information processing goes on in the cerebral cortex, the outer layer of the brain. The basic organizational unit appears to be a column of tissue about 0.5 mm in diameter, containing about 20,000 neurons and extending the full depth of the cortex about 4 mm in humans).

We now have some data on the mapping between areas of the brain and the parts of the body that they control or from which they receive sensory input. Such mappings are able to change radically over the course of a few weeks, and some animals seem to have multiple maps. Moreover, we do not fully understand how other areas can take over functions when one area is damaged. There is almost no theory on how an individual memory is stored.

The measurement of intact brain activity began in 1929 with the invention by Hans Berger of the electroencephalograph (EEG). The recent development of functional magnetic resonance imaging (fMRI) (Ogawa *et al.*, 1990; Cabeza and Nyberg, 2001) is giving neuroscientists unprecedentedly detailed images of brain activity, enabling measurements that correspond in interesting ways to ongoing cognitive processes. These are augmented by advances in single-cell recording of neuron activity. Individual neurons can be stimulated electrically, chemically, or even optically (Han and Boyden, 2007), allowing neuronal input–output relationships to be mapped. Despite these advances, we are still a long way from understanding how cognitive processes actually work.

The truly amazing conclusion is that *a collection of simple cells can lead to thought, action, and consciousness* or, in the pithy words of John Searle (1992), *brains cause minds*.



	Supercomputer	Personal Computer	Human Brain
Computational units	10 ⁴ CPUs, 10 ¹² transistors	4 CPUs, 10 ⁹ transistors	10 ¹¹ neurons
Storage units	10 ¹⁴ bits RAM 10 ¹⁵ bits disk	10 ¹¹ bits RAM 10 ¹³ bits disk	10 ¹¹ neurons 10 ¹⁴ synapses
Cycle time	10 ⁻⁹ sec	10 ⁻⁹ sec	10 ⁻³ sec
Operations/sec	10 ¹⁵	10 ¹⁰	10 ¹⁷
Memory updates/sec	10 ¹⁴	10 ¹⁰	10 ¹⁴

Figure 1.3 A crude comparison of the raw computational resources available to the IBM BLUE GENE supercomputer, a typical personal computer of 2008, and the human brain. The brain's numbers are essentially fixed, whereas the supercomputer's numbers have been increasing by a factor of 10 every 5 years or so, allowing it to achieve rough parity with the brain. The personal computer lags behind on all metrics except cycle time.

The only real alternative theory is mysticism: that minds operate in some mystical realm that is beyond physical science.

Brains and digital computers have somewhat different properties. Figure 1.3 shows that computers have a cycle time that is a million times faster than a brain. The brain makes up for that with far more storage and interconnection than even a high-end personal computer, although the largest supercomputers have a capacity that is similar to the brain's. (It should be noted, however, that the brain does not seem to use all of its neurons simultaneously.) Futurists make much of these numbers, pointing to an approaching **singularity** at which computers reach a superhuman level of performance (Vinge, 1993; Kurzweil, 2005), but the raw comparisons are not especially informative. Even with a computer of virtually unlimited capacity, we still would not know how to achieve the brain's level of intelligence.

SINGULARITY

1.2.5 Psychology

- How do humans and animals think and act?

The origins of scientific psychology are usually traced to the work of the German physicist Hermann von Helmholtz (1821–1894) and his student Wilhelm Wundt (1832–1920). Helmholtz applied the scientific method to the study of human vision, and his *Handbook of Physiological Optics* is even now described as “the single most important treatise on the physics and physiology of human vision” (Nalwa, 1993, p.15). In 1879, Wundt opened the first laboratory of experimental psychology, at the University of Leipzig. Wundt insisted on carefully controlled experiments in which his workers would perform a perceptual or associative task while introspecting on their thought processes. The careful controls went a long way toward making psychology a science, but the subjective nature of the data made it unlikely that an experimenter would ever disconfirm his or her own theories. Biologists studying animal behavior, on the other hand, lacked introspective data and developed an objective methodology, as described by H. S. Jennings (1906) in his influential work *Behavior of the Lower Organisms*. Applying this viewpoint to humans, the **behaviorism** movement, led by John Watson (1878–1958), rejected *any* theory involving mental processes on the grounds

BEHAVIORISM

that introspection could not provide reliable evidence. Behaviorists insisted on studying only objective measures of the percepts (or *stimulus*) given to an animal and its resulting actions (or *response*). Behaviorism discovered a lot about rats and pigeons but had less success at understanding humans.

Cognitive psychology, which views the brain as an information-processing device, can be traced back at least to the works of William James (1842–1910). Helmholtz also insisted that perception involved a form of unconscious logical inference. The cognitive viewpoint was largely eclipsed by behaviorism in the United States, but at Cambridge’s Applied Psychology Unit, directed by Frederic Bartlett (1886–1969), cognitive modeling was able to flourish. *The Nature of Explanation*, by Bartlett’s student and successor Kenneth Craik (1943), forcefully reestablished the legitimacy of such “mental” terms as beliefs and goals, arguing that they are just as scientific as, say, using pressure and temperature to talk about gases, despite their being made of molecules that have neither. Craik specified the three key steps of a knowledge-based agent: (1) the stimulus must be translated into an internal representation, (2) the representation is manipulated by cognitive processes to derive new internal representations, and (3) these are in turn retranslated back into action. He clearly explained why this was a good design for an agent:

If the organism carries a “small-scale model” of external reality and of its own possible actions within its head, it is able to try out various alternatives, conclude which is the best of them, react to future situations before they arise, utilize the knowledge of past events in dealing with the present and future, and in every way to react in a much fuller, safer, and more competent manner to the emergencies which face it. (Craik, 1943)

After Craik’s death in a bicycle accident in 1945, his work was continued by Donald Broadbent, whose book *Perception and Communication* (1958) was one of the first works to model psychological phenomena as information processing. Meanwhile, in the United States, the development of computer modeling led to the creation of the field of **cognitive science**. The field can be said to have started at a workshop in September 1956 at MIT. (We shall see that this is just two months after the conference at which AI itself was “born.”) At the workshop, George Miller presented *The Magic Number Seven*, Noam Chomsky presented *Three Models of Language*, and Allen Newell and Herbert Simon presented *The Logic Theory Machine*. These three influential papers showed how computer models could be used to address the psychology of memory, language, and logical thinking, respectively. It is now a common (although far from universal) view among psychologists that “a cognitive theory should be like a computer program” (Anderson, 1980); that is, it should describe a detailed information-processing mechanism whereby some cognitive function might be implemented.

1.2.6 Computer engineering

- How can we build an efficient computer?

For artificial intelligence to succeed, we need two things: intelligence and an artifact. The computer has been the artifact of choice. The modern digital electronic computer was invented independently and almost simultaneously by scientists in three countries embattled in

World War II. The first *operational* computer was the electromechanical Heath Robinson,⁸ built in 1940 by Alan Turing's team for a single purpose: deciphering German messages. In 1943, the same group developed the Colossus, a powerful general-purpose machine based on vacuum tubes.⁹ The first operational *programmable* computer was the Z-3, the invention of Konrad Zuse in Germany in 1941. Zuse also invented floating-point numbers and the first high-level programming language, Plankalkül. The first *electronic* computer, the ABC, was assembled by John Atanasoff and his student Clifford Berry between 1940 and 1942 at Iowa State University. Atanasoff's research received little support or recognition; it was the ENIAC, developed as part of a secret military project at the University of Pennsylvania by a team including John Mauchly and John Eckert, that proved to be the most influential forerunner of modern computers.

Since that time, each generation of computer hardware has brought an increase in speed and capacity and a decrease in price. Performance doubled every 18 months or so until around 2005, when power dissipation problems led manufacturers to start multiplying the number of CPU cores rather than the clock speed. Current expectations are that future increases in power will come from massive parallelism—a curious convergence with the properties of the brain.

Of course, there were calculating devices before the electronic computer. The earliest automated machines, dating from the 17th century, were discussed on page 6. The first *programmable* machine was a loom, devised in 1805 by Joseph Marie Jacquard (1752–1834), that used punched cards to store instructions for the pattern to be woven. In the mid-19th century, Charles Babbage (1792–1871) designed two machines, neither of which he completed. The Difference Engine was intended to compute mathematical tables for engineering and scientific projects. It was finally built and shown to work in 1991 at the Science Museum in London (Swade, 2000). Babbage's Analytical Engine was far more ambitious: it included addressable memory, stored programs, and conditional jumps and was the first artifact capable of universal computation. Babbage's colleague Ada Lovelace, daughter of the poet Lord Byron, was perhaps the world's first programmer. (The programming language Ada is named after her.) She wrote programs for the unfinished Analytical Engine and even speculated that the machine could play chess or compose music.

AI also owes a debt to the software side of computer science, which has supplied the operating systems, programming languages, and tools needed to write modern programs (and papers about them). But this is one area where the debt has been repaid: work in AI has pioneered many ideas that have made their way back to mainstream computer science, including time sharing, interactive interpreters, personal computers with windows and mice, rapid development environments, the linked list data type, automatic storage management, and key concepts of symbolic, functional, declarative, and object-oriented programming.

⁸ Heath Robinson was a cartoonist famous for his depictions of whimsical and absurdly complicated contraptions for everyday tasks such as buttering toast.

⁹ In the postwar period, Turing wanted to use these computers for AI research—for example, one of the first chess programs (Turing *et al.*, 1953). His efforts were blocked by the British government.

1.2.7 Control theory and cybernetics

- How can artifacts operate under their own control?

Ktesibios of Alexandria (c. 250 B.C.) built the first self-controlling machine: a water clock with a regulator that maintained a constant flow rate. This invention changed the definition of what an artifact could do. Previously, only living things could modify their behavior in response to changes in the environment. Other examples of self-regulating feedback control systems include the steam engine governor, created by James Watt (1736–1819), and the thermostat, invented by Cornelis Drebbel (1572–1633), who also invented the submarine. The mathematical theory of stable feedback systems was developed in the 19th century.

CONTROL THEORY

The central figure in the creation of what is now called **control theory** was Norbert Wiener (1894–1964). Wiener was a brilliant mathematician who worked with Bertrand Russell, among others, before developing an interest in biological and mechanical control systems and their connection to cognition. Like Craik (who also used control systems as psychological models), Wiener and his colleagues Arturo Rosenblueth and Julian Bigelow challenged the behaviorist orthodoxy (Rosenblueth *et al.*, 1943). They viewed purposive behavior as arising from a regulatory mechanism trying to minimize “error”—the difference between current state and goal state. In the late 1940s, Wiener, along with Warren McCulloch, Walter Pitts, and John von Neumann, organized a series of influential conferences that explored the new mathematical and computational models of cognition. Wiener’s book *Cybernetics* (1948) became a bestseller and awoke the public to the possibility of artificially intelligent machines. Meanwhile, in Britain, W. Ross Ashby (Ashby, 1940) pioneered similar ideas. Ashby, Alan Turing, Grey Walter, and others formed the Ratio Club for “those who had Wiener’s ideas before Wiener’s book appeared.” Ashby’s *Design for a Brain* (1948, 1952) elaborated on his idea that intelligence could be created by the use of **homeostatic** devices containing appropriate feedback loops to achieve stable adaptive behavior.

CYBERNETICS

HOMEOSTATIC

OBJECTIVE
FUNCTION

Modern control theory, especially the branch known as stochastic optimal control, has as its goal the design of systems that maximize an **objective function** over time. This roughly matches our view of AI: designing systems that behave optimally. Why, then, are AI and control theory two different fields, despite the close connections among their founders? The answer lies in the close coupling between the mathematical techniques that were familiar to the participants and the corresponding sets of problems that were encompassed in each world view. Calculus and matrix algebra, the tools of control theory, lend themselves to systems that are describable by fixed sets of continuous variables, whereas AI was founded in part as a way to escape from these perceived limitations. The tools of logical inference and computation allowed AI researchers to consider problems such as language, vision, and planning that fell completely outside the control theorist’s purview.

1.2.8 Linguistics

- How does language relate to thought?

In 1957, B. F. Skinner published *Verbal Behavior*. This was a comprehensive, detailed account of the behaviorist approach to language learning, written by the foremost expert in

the field. But curiously, a review of the book became as well known as the book itself, and served to almost kill off interest in behaviorism. The author of the review was the linguist Noam Chomsky, who had just published a book on his own theory, *Syntactic Structures*. Chomsky pointed out that the behaviorist theory did not address the notion of creativity in language—it did not explain how a child could understand and make up sentences that he or she had never heard before. Chomsky’s theory—based on syntactic models going back to the Indian linguist Panini (c. 350 B.C.)—could explain this, and unlike previous theories, it was formal enough that it could in principle be programmed.

COMPUTATIONAL
LINGUISTICS

Modern linguistics and AI, then, were “born” at about the same time, and grew up together, intersecting in a hybrid field called **computational linguistics** or **natural language processing**. The problem of understanding language soon turned out to be considerably more complex than it seemed in 1957. Understanding language requires an understanding of the subject matter and context, not just an understanding of the structure of sentences. This might seem obvious, but it was not widely appreciated until the 1960s. Much of the early work in **knowledge representation** (the study of how to put knowledge into a form that a computer can reason with) was tied to language and informed by research in linguistics, which was connected in turn to decades of work on the philosophical analysis of language.

1.3 THE HISTORY OF ARTIFICIAL INTELLIGENCE

With the background material behind us, we are ready to cover the development of AI itself.

1.3.1 The gestation of artificial intelligence (1943–1955)

The first work that is now generally recognized as AI was done by Warren McCulloch and Walter Pitts (1943). They drew on three sources: knowledge of the basic physiology and function of neurons in the brain; a formal analysis of propositional logic due to Russell and Whitehead; and Turing’s theory of computation. They proposed a model of artificial neurons in which each neuron is characterized as being “on” or “off,” with a switch to “on” occurring in response to stimulation by a sufficient number of neighboring neurons. The state of a neuron was conceived of as “factually equivalent to a proposition which proposed its adequate stimulus.” They showed, for example, that any computable function could be computed by some network of connected neurons, and that all the logical connectives (and, or, not, etc.) could be implemented by simple net structures. McCulloch and Pitts also suggested that suitably defined networks could learn. Donald Hebb (1949) demonstrated a simple updating rule for modifying the connection strengths between neurons. His rule, now called **Hebbian learning**, remains an influential model to this day.

HEBBIAN LEARNING

Two undergraduate students at Harvard, Marvin Minsky and Dean Edmonds, built the first neural network computer in 1950. The SNARC, as it was called, used 3000 vacuum tubes and a surplus automatic pilot mechanism from a B-24 bomber to simulate a network of 40 neurons. Later, at Princeton, Minsky studied universal computation in neural networks. His Ph.D. committee was skeptical about whether this kind of work should be considered

mathematics, but von Neumann reportedly said, “If it isn’t now, it will be someday.” Minsky was later to prove influential theorems showing the limitations of neural network research.

There were a number of early examples of work that can be characterized as AI, but Alan Turing’s vision was perhaps the most influential. He gave lectures on the topic as early as 1947 at the London Mathematical Society and articulated a persuasive agenda in his 1950 article “Computing Machinery and Intelligence.” Therein, he introduced the Turing Test, machine learning, genetic algorithms, and reinforcement learning. He proposed the *Child Programme* idea, explaining “Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulated the child’s?”

1.3.2 The birth of artificial intelligence (1956)

Princeton was home to another influential figure in AI, John McCarthy. After receiving his PhD there in 1951 and working for two years as an instructor, McCarthy moved to Stanford and then to Dartmouth College, which was to become the official birthplace of the field. McCarthy convinced Minsky, Claude Shannon, and Nathaniel Rochester to help him bring together U.S. researchers interested in automata theory, neural nets, and the study of intelligence. They organized a two-month workshop at Dartmouth in the summer of 1956. The proposal states:¹⁰

We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

There were 10 attendees in all, including Trenchard More from Princeton, Arthur Samuel from IBM, and Ray Solomonoff and Oliver Selfridge from MIT.

Two researchers from Carnegie Tech,¹¹ Allen Newell and Herbert Simon, rather stole the show. Although the others had ideas and in some cases programs for particular applications such as checkers, Newell and Simon already had a reasoning program, the Logic Theorist (LT), about which Simon claimed, “We have invented a computer program capable of thinking non-numerically, and thereby solved the venerable mind–body problem.”¹² Soon after the workshop, the program was able to prove most of the theorems in Chapter 2 of Rus-

¹⁰ This was the first official usage of McCarthy’s term *artificial intelligence*. Perhaps “computational rationality” would have been more precise and less threatening, but “AI” has stuck. At the 50th anniversary of the Dartmouth conference, McCarthy stated that he resisted the terms “computer” or “computational” in deference to Norbert Wiener, who was promoting analog cybernetic devices rather than digital computers.

¹¹ Now Carnegie Mellon University (CMU).

¹² Newell and Simon also invented a list-processing language, IPL, to write LT. They had no compiler and translated it into machine code by hand. To avoid errors, they worked in parallel, calling out binary numbers to each other as they wrote each instruction to make sure they agreed.

sell and Whitehead's *Principia Mathematica*. Russell was reportedly delighted when Simon showed him that the program had come up with a proof for one theorem that was shorter than the one in *Principia*. The editors of the *Journal of Symbolic Logic* were less impressed; they rejected a paper coauthored by Newell, Simon, and Logic Theorist.

The Dartmouth workshop did not lead to any new breakthroughs, but it did introduce all the major figures to each other. For the next 20 years, the field would be dominated by these people and their students and colleagues at MIT, CMU, Stanford, and IBM.

Looking at the proposal for the Dartmouth workshop (McCarthy *et al.*, 1955), we can see why it was necessary for AI to become a separate field. Why couldn't all the work done in AI have taken place under the name of control theory or operations research or decision theory, which, after all, have objectives similar to those of AI? Or why isn't AI a branch of mathematics? The first answer is that AI from the start embraced the idea of duplicating human faculties such as creativity, self-improvement, and language use. None of the other fields were addressing these issues. The second answer is methodology. AI is the only one of these fields that is clearly a branch of computer science (although operations research does share an emphasis on computer simulations), and AI is the only field to attempt to build machines that will function autonomously in complex, changing environments.

1.3.3 Early enthusiasm, great expectations (1952–1969)

The early years of AI were full of successes—in a limited way. Given the primitive computers and programming tools of the time and the fact that only a few years earlier computers were seen as things that could do arithmetic and no more, it was astonishing whenever a computer did anything remotely clever. The intellectual establishment, by and large, preferred to believe that “a machine can never do *X*.” (See Chapter 26 for a long list of *X*'s gathered by Turing.) AI researchers naturally responded by demonstrating one *X* after another. John McCarthy referred to this period as the “Look, Ma, no hands!” era.

Newell and Simon's early success was followed up with the General Problem Solver, or GPS. Unlike Logic Theorist, this program was designed from the start to imitate human problem-solving protocols. Within the limited class of puzzles it could handle, it turned out that the order in which the program considered subgoals and possible actions was similar to that in which humans approached the same problems. Thus, GPS was probably the first program to embody the “thinking humanly” approach. The success of GPS and subsequent programs as models of cognition led Newell and Simon (1976) to formulate the famous **physical symbol system** hypothesis, which states that “a physical symbol system has the necessary and sufficient means for general intelligent action.” What they meant is that any system (human or machine) exhibiting intelligence must operate by manipulating data structures composed of symbols. We will see later that this hypothesis has been challenged from many directions.

At IBM, Nathaniel Rochester and his colleagues produced some of the first AI programs. Herbert Gelernter (1959) constructed the Geometry Theorem Prover, which was able to prove theorems that many students of mathematics would find quite tricky. Starting in 1952, Arthur Samuel wrote a series of programs for checkers (draughts) that eventually learned to play at a strong amateur level. Along the way, he disproved the idea that comput-

ers can do only what they are told to: his program quickly learned to play a better game than its creator. The program was demonstrated on television in February 1956, creating a strong impression. Like Turing, Samuel had trouble finding computer time. Working at night, he used machines that were still on the testing floor at IBM's manufacturing plant. Chapter 5 covers game playing, and Chapter 21 explains the learning techniques used by Samuel.

LISP

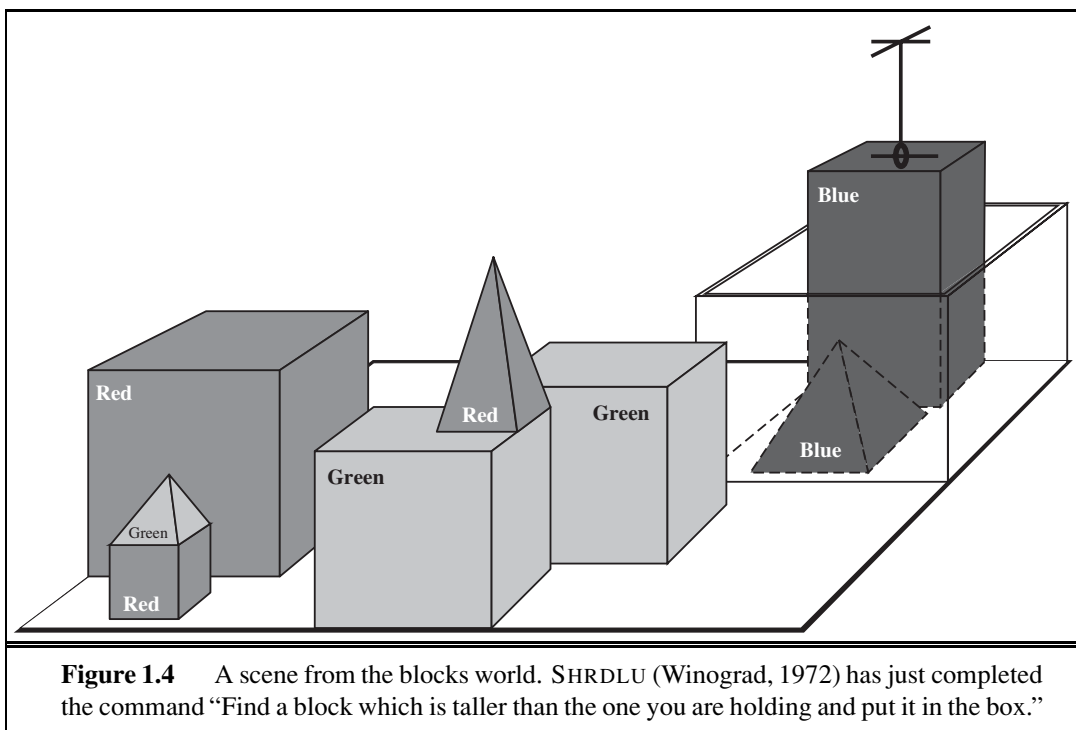
John McCarthy moved from Dartmouth to MIT and there made three crucial contributions in one historic year: 1958. In MIT AI Lab Memo No. 1, McCarthy defined the high-level language **Lisp**, which was to become the dominant AI programming language for the next 30 years. With Lisp, McCarthy had the tool he needed, but access to scarce and expensive computing resources was also a serious problem. In response, he and others at MIT invented time sharing. Also in 1958, McCarthy published a paper entitled *Programs with Common Sense*, in which he described the Advice Taker, a hypothetical program that can be seen as the first complete AI system. Like the Logic Theorist and Geometry Theorem Prover, McCarthy's program was designed to use knowledge to search for solutions to problems. But unlike the others, it was to embody general knowledge of the world. For example, he showed how some simple axioms would enable the program to generate a plan to drive to the airport. The program was also designed to accept new axioms in the normal course of operation, thereby allowing it to achieve competence in new areas *without being reprogrammed*. The Advice Taker thus embodied the central principles of knowledge representation and reasoning: that it is useful to have a formal, explicit representation of the world and its workings and to be able to manipulate that representation with deductive processes. It is remarkable how much of the 1958 paper remains relevant today.

1958 also marked the year that Marvin Minsky moved to MIT. His initial collaboration with McCarthy did not last, however. McCarthy stressed representation and reasoning in formal logic, whereas Minsky was more interested in getting programs to work and eventually developed an anti-logic outlook. In 1963, McCarthy started the AI lab at Stanford. His plan to use logic to build the ultimate Advice Taker was advanced by J. A. Robinson's discovery in 1965 of the resolution method (a complete theorem-proving algorithm for first-order logic; see Chapter 9). Work at Stanford emphasized general-purpose methods for logical reasoning. Applications of logic included Cordell Green's question-answering and planning systems (Green, 1969b) and the Shakey robotics project at the Stanford Research Institute (SRI). The latter project, discussed further in Chapter 25, was the first to demonstrate the complete integration of logical reasoning and physical activity.

MICROWORLD

Minsky supervised a series of students who chose limited problems that appeared to require intelligence to solve. These limited domains became known as **microworlds**. James Slagle's SAINT program (1963) was able to solve closed-form calculus integration problems typical of first-year college courses. Tom Evans's ANALOGY program (1968) solved geometric analogy problems that appear in IQ tests. Daniel Bobrow's STUDENT program (1967) solved algebra story problems, such as the following:

If the number of customers Tom gets is twice the square of 20 percent of the number of advertisements he runs, and the number of advertisements he runs is 45, what is the number of customers Tom gets?



The most famous microworld was the blocks world, which consists of a set of solid blocks placed on a tabletop (or more often, a simulation of a tabletop), as shown in Figure 1.4. A typical task in this world is to rearrange the blocks in a certain way, using a robot hand that can pick up one block at a time. The blocks world was home to the vision project of David Huffman (1971), the vision and constraint-propagation work of David Waltz (1975), the learning theory of Patrick Winston (1970), the natural-language-understanding program of Terry Winograd (1972), and the planner of Scott Fahlman (1974).

Early work building on the neural networks of McCulloch and Pitts also flourished. The work of Winograd and Cowan (1963) showed how a large number of elements could collectively represent an individual concept, with a corresponding increase in robustness and parallelism. Hebb’s learning methods were enhanced by Bernie Widrow (Widrow and Hoff, 1960; Widrow, 1962), who called his networks **adalines**, and by Frank Rosenblatt (1962) with his **perceptrons**. The **perceptron convergence theorem** (Block *et al.*, 1962) says that the learning algorithm can adjust the connection strengths of a perceptron to match any input data, provided such a match exists. These topics are covered in Chapter 20.

1.3.4 A dose of reality (1966–1973)

From the beginning, AI researchers were not shy about making predictions of their coming successes. The following statement by Herbert Simon in 1957 is often quoted:

It is not my aim to surprise or shock you—but the simplest way I can summarize is to say that there are now in the world machines that think, that learn and that create. Moreover,

their ability to do these things is going to increase rapidly until—in a visible future—the range of problems they can handle will be coextensive with the range to which the human mind has been applied.

Terms such as “visible future” can be interpreted in various ways, but Simon also made more concrete predictions: that within 10 years a computer would be chess champion, and a significant mathematical theorem would be proved by machine. These predictions came true (or approximately true) within 40 years rather than 10. Simon’s overconfidence was due to the promising performance of early AI systems on simple examples. In almost all cases, however, these early systems turned out to fail miserably when tried out on wider selections of problems and on more difficult problems.

The first kind of difficulty arose because most early programs knew nothing of their subject matter; they succeeded by means of simple syntactic manipulations. A typical story occurred in early machine translation efforts, which were generously funded by the U.S. National Research Council in an attempt to speed up the translation of Russian scientific papers in the wake of the Sputnik launch in 1957. It was thought initially that simple syntactic transformations based on the grammars of Russian and English, and word replacement from an electronic dictionary, would suffice to preserve the exact meanings of sentences. The fact is that accurate translation requires background knowledge in order to resolve ambiguity and establish the content of the sentence. The famous retranslation of “the spirit is willing but the flesh is weak” as “the vodka is good but the meat is rotten” illustrates the difficulties encountered. In 1966, a report by an advisory committee found that “there has been no machine translation of general scientific text, and none is in immediate prospect.” All U.S. government funding for academic translation projects was canceled. Today, machine translation is an imperfect but widely used tool for technical, commercial, government, and Internet documents.

The second kind of difficulty was the intractability of many of the problems that AI was attempting to solve. Most of the early AI programs solved problems by trying out different combinations of steps until the solution was found. This strategy worked initially because microworlds contained very few objects and hence very few possible actions and very short solution sequences. Before the theory of computational complexity was developed, it was widely thought that “scaling up” to larger problems was simply a matter of faster hardware and larger memories. The optimism that accompanied the development of resolution theorem proving, for example, was soon dampened when researchers failed to prove theorems involving more than a few dozen facts. *The fact that a program can find a solution in principle does not mean that the program contains any of the mechanisms needed to find it in practice.*

The illusion of unlimited computational power was not confined to problem-solving programs. Early experiments in **machine evolution** (now called **genetic algorithms**) (Friedberg, 1958; Friedberg *et al.*, 1959) were based on the undoubtedly correct belief that by making an appropriate series of small mutations to a machine-code program, one can generate a program with good performance for any particular task. The idea, then, was to try random mutations with a selection process to preserve mutations that seemed useful. Despite thousands of hours of CPU time, almost no progress was demonstrated. Modern genetic algorithms use better representations and have shown more success.



MACHINE EVOLUTION
GENETIC
ALGORITHM

Failure to come to grips with the “combinatorial explosion” was one of the main criticisms of AI contained in the Lighthill report (Lighthill, 1973), which formed the basis for the decision by the British government to end support for AI research in all but two universities. (Oral tradition paints a somewhat different and more colorful picture, with political ambitions and personal animosities whose description is beside the point.)

A third difficulty arose because of some fundamental limitations on the basic structures being used to generate intelligent behavior. For example, Minsky and Papert’s book *Perceptrons* (1969) proved that, although perceptrons (a simple form of neural network) could be shown to learn anything they were capable of representing, they could represent very little. In particular, a two-input perceptron (restricted to be simpler than the form Rosenblatt originally studied) could not be trained to recognize when its two inputs were different. Although their results did not apply to more complex, multilayer networks, research funding for neural-net research soon dwindled to almost nothing. Ironically, the new back-propagation learning algorithms for multilayer networks that were to cause an enormous resurgence in neural-net research in the late 1980s were actually discovered first in 1969 (Bryson and Ho, 1969).

1.3.5 Knowledge-based systems: The key to power? (1969–1979)

The picture of problem solving that had arisen during the first decade of AI research was of a general-purpose search mechanism trying to string together elementary reasoning steps to find complete solutions. Such approaches have been called **weak methods** because, although general, they do not scale up to large or difficult problem instances. The alternative to weak methods is to use more powerful, domain-specific knowledge that allows larger reasoning steps and can more easily handle typically occurring cases in narrow areas of expertise. One might say that to solve a hard problem, you have to almost know the answer already.

The DENDRAL program (Buchanan *et al.*, 1969) was an early example of this approach. It was developed at Stanford, where Ed Feigenbaum (a former student of Herbert Simon), Bruce Buchanan (a philosopher turned computer scientist), and Joshua Lederberg (a Nobel laureate geneticist) teamed up to solve the problem of inferring molecular structure from the information provided by a mass spectrometer. The input to the program consists of the elementary formula of the molecule (e.g., $C_6H_{13}NO_2$) and the mass spectrum giving the masses of the various fragments of the molecule generated when it is bombarded by an electron beam. For example, the mass spectrum might contain a peak at $m = 15$, corresponding to the mass of a methyl (CH_3) fragment.

The naive version of the program generated all possible structures consistent with the formula, and then predicted what mass spectrum would be observed for each, comparing this with the actual spectrum. As one might expect, this is intractable for even moderate-sized molecules. The DENDRAL researchers consulted analytical chemists and found that they worked by looking for well-known patterns of peaks in the spectrum that suggested common substructures in the molecule. For example, the following rule is used to recognize a ketone ($C=O$) subgroup (which weighs 28):

if there are two peaks at x_1 and x_2 such that

(a) $x_1 + x_2 = M + 28$ (M is the mass of the whole molecule);

- (b) $x_1 - 28$ is a high peak;
 - (c) $x_2 - 28$ is a high peak;
 - (d) At least one of x_1 and x_2 is high.
- then** there is a ketone subgroup

Recognizing that the molecule contains a particular substructure reduces the number of possible candidates enormously. DENDRAL was powerful because

All the relevant theoretical knowledge to solve these problems has been mapped over from its general form in the [spectrum prediction component] (“first principles”) to efficient special forms (“cookbook recipes”). (Feigenbaum *et al.*, 1971)

The significance of DENDRAL was that it was the first successful *knowledge-intensive* system: its expertise derived from large numbers of special-purpose rules. Later systems also incorporated the main theme of McCarthy’s Advice Taker approach—the clean separation of the knowledge (in the form of rules) from the reasoning component.

EXPERT SYSTEMS

With this lesson in mind, Feigenbaum and others at Stanford began the Heuristic Programming Project (HPP) to investigate the extent to which the new methodology of **expert systems** could be applied to other areas of human expertise. The next major effort was in the area of medical diagnosis. Feigenbaum, Buchanan, and Dr. Edward Shortliffe developed MYCIN to diagnose blood infections. With about 450 rules, MYCIN was able to perform as well as some experts, and considerably better than junior doctors. It also contained two major differences from DENDRAL. First, unlike the DENDRAL rules, no general theoretical model existed from which the MYCIN rules could be deduced. They had to be acquired from extensive interviewing of experts, who in turn acquired them from textbooks, other experts, and direct experience of cases. Second, the rules had to reflect the uncertainty associated with medical knowledge. MYCIN incorporated a calculus of uncertainty called **certainty factors** (see Chapter 14), which seemed (at the time) to fit well with how doctors assessed the impact of evidence on the diagnosis.

CERTAINTY FACTOR

The importance of domain knowledge was also apparent in the area of understanding natural language. Although Winograd’s SHRDLU system for understanding natural language had engendered a good deal of excitement, its dependence on syntactic analysis caused some of the same problems as occurred in the early machine translation work. It was able to overcome ambiguity and understand pronoun references, but this was mainly because it was designed specifically for one area—the blocks world. Several researchers, including Eugene Charniak, a fellow graduate student of Winograd’s at MIT, suggested that robust language understanding would require general knowledge about the world and a general method for using that knowledge.

At Yale, linguist-turned-AI-researcher Roger Schank emphasized this point, claiming, “There is no such thing as syntax,” which upset a lot of linguists but did serve to start a useful discussion. Schank and his students built a series of programs (Schank and Abelson, 1977; Wilensky, 1978; Schank and Riesbeck, 1981; Dyer, 1983) that all had the task of understanding natural language. The emphasis, however, was less on language *per se* and more on the problems of representing and reasoning with the knowledge required for language understanding. The problems included representing stereotypical situations (Cullingford, 1981),

describing human memory organization (Rieger, 1976; Kolodner, 1983), and understanding plans and goals (Wilensky, 1983).

FRAMES

The widespread growth of applications to real-world problems caused a concurrent increase in the demands for workable knowledge representation schemes. A large number of different representation and reasoning languages were developed. Some were based on logic—for example, the Prolog language became popular in Europe, and the PLANNER family in the United States. Others, following Minsky’s idea of **frames** (1975), adopted a more structured approach, assembling facts about particular object and event types and arranging the types into a large taxonomic hierarchy analogous to a biological taxonomy.

1.3.6 AI becomes an industry (1980–present)

The first successful commercial expert system, R1, began operation at the Digital Equipment Corporation (McDermott, 1982). The program helped configure orders for new computer systems; by 1986, it was saving the company an estimated \$40 million a year. By 1988, DEC’s AI group had 40 expert systems deployed, with more on the way. DuPont had 100 in use and 500 in development, saving an estimated \$10 million a year. Nearly every major U.S. corporation had its own AI group and was either using or investigating expert systems.

In 1981, the Japanese announced the “Fifth Generation” project, a 10-year plan to build intelligent computers running Prolog. In response, the United States formed the Microelectronics and Computer Technology Corporation (MCC) as a research consortium designed to assure national competitiveness. In both cases, AI was part of a broad effort, including chip design and human-interface research. In Britain, the Alvey report reinstated the funding that was cut by the Lighthill report.¹³ In all three countries, however, the projects never met their ambitious goals.

Overall, the AI industry boomed from a few million dollars in 1980 to billions of dollars in 1988, including hundreds of companies building expert systems, vision systems, robots, and software and hardware specialized for these purposes. Soon after that came a period called the “AI Winter,” in which many companies fell by the wayside as they failed to deliver on extravagant promises.

1.3.7 The return of neural networks (1986–present)

BACK-PROPAGATION

In the mid-1980s at least four different groups reinvented the **back-propagation** learning algorithm first found in 1969 by Bryson and Ho. The algorithm was applied to many learning problems in computer science and psychology, and the widespread dissemination of the results in the collection *Parallel Distributed Processing* (Rumelhart and McClelland, 1986) caused great excitement.

CONNECTIONIST

These so-called **connectionist** models of intelligent systems were seen by some as direct competitors both to the symbolic models promoted by Newell and Simon and to the logicist approach of McCarthy and others (Smolensky, 1988). It might seem obvious that at some level humans manipulate symbols—in fact, Terrence Deacon’s book *The Symbolic*

¹³ To save embarrassment, a new field called IKBS (Intelligent Knowledge-Based Systems) was invented because Artificial Intelligence had been officially canceled.

Species (1997) suggests that this is the *defining characteristic* of humans—but the most ardent connectionists questioned whether symbol manipulation had any real explanatory role in detailed models of cognition. This question remains unanswered, but the current view is that connectionist and symbolic approaches are complementary, not competing. As occurred with the separation of AI and cognitive science, modern neural network research has bifurcated into two fields, one concerned with creating effective network architectures and algorithms and understanding their mathematical properties, the other concerned with careful modeling of the empirical properties of actual neurons and ensembles of neurons.

1.3.8 AI adopts the scientific method (1987–present)

Recent years have seen a revolution in both the content and the methodology of work in artificial intelligence.¹⁴ It is now more common to build on existing theories than to propose brand-new ones, to base claims on rigorous theorems or hard experimental evidence rather than on intuition, and to show relevance to real-world applications rather than toy examples.

AI was founded in part as a rebellion against the limitations of existing fields like control theory and statistics, but now it is embracing those fields. As David McAllester (1998) put it:

In the early period of AI it seemed plausible that new forms of symbolic computation, e.g., frames and semantic networks, made much of classical theory obsolete. This led to a form of isolationism in which AI became largely separated from the rest of computer science. This isolationism is currently being abandoned. There is a recognition that machine learning should not be isolated from information theory, that uncertain reasoning should not be isolated from stochastic modeling, that search should not be isolated from classical optimization and control, and that automated reasoning should not be isolated from formal methods and static analysis.

In terms of methodology, AI has finally come firmly under the scientific method. To be accepted, hypotheses must be subjected to rigorous empirical experiments, and the results must be analyzed statistically for their importance (Cohen, 1995). It is now possible to replicate experiments by using shared repositories of test data and code.

The field of speech recognition illustrates the pattern. In the 1970s, a wide variety of different architectures and approaches were tried. Many of these were rather *ad hoc* and fragile, and were demonstrated on only a few specially selected examples. In recent years, approaches based on **hidden Markov models** (HMMs) have come to dominate the area. Two aspects of HMMs are relevant. First, they are based on a rigorous mathematical theory. This has allowed speech researchers to build on several decades of mathematical results developed in other fields. Second, they are generated by a process of training on a large corpus of real speech data. This ensures that the performance is robust, and in rigorous blind tests the HMMs have been improving their scores steadily. Speech technology and the related field of handwritten character recognition are already making the transition to widespread industrial

¹⁴ Some have characterized this change as a victory of the **neats**—those who think that AI theories should be grounded in mathematical rigor—over the **scruffies**—those who would rather try out lots of ideas, write some programs, and then assess what seems to be working. Both approaches are important. A shift toward neatness implies that the field has reached a level of stability and maturity. Whether that stability will be disrupted by a new scruffy idea is another question.

and consumer applications. Note that there is no scientific claim that humans use HMMs to recognize speech; rather, HMMs provide a mathematical framework for understanding the problem and support the engineering claim that they work well in practice.

Machine translation follows the same course as speech recognition. In the 1950s there was initial enthusiasm for an approach based on sequences of words, with models learned according to the principles of information theory. That approach fell out of favor in the 1960s, but returned in the late 1990s and now dominates the field.

Neural networks also fit this trend. Much of the work on neural nets in the 1980s was done in an attempt to scope out what could be done and to learn how neural nets differ from “traditional” techniques. Using improved methodology and theoretical frameworks, the field arrived at an understanding in which neural nets can now be compared with corresponding techniques from statistics, pattern recognition, and machine learning, and the most promising technique can be applied to each application. As a result of these developments, so-called **data mining** technology has spawned a vigorous new industry.

DATA MINING

Judea Pearl’s (1988) *Probabilistic Reasoning in Intelligent Systems* led to a new acceptance of probability and decision theory in AI, following a resurgence of interest epitomized by Peter Cheeseman’s (1985) article “In Defense of Probability.” The **Bayesian network** formalism was invented to allow efficient representation of, and rigorous reasoning with, uncertain knowledge. This approach largely overcomes many problems of the probabilistic reasoning systems of the 1960s and 1970s; it now dominates AI research on uncertain reasoning and expert systems. The approach allows for learning from experience, and it combines the best of classical AI and neural nets. Work by Judea Pearl (1982a) and by Eric Horvitz and David Heckerman (Horvitz and Heckerman, 1986; Horvitz *et al.*, 1986) promoted the idea of *normative* expert systems: ones that act rationally according to the laws of decision theory and do not try to imitate the thought steps of human experts. The WindowsTM operating system includes several normative diagnostic expert systems for correcting problems. Chapters 13 to 16 cover this area.

BAYESIAN NETWORK

Similar gentle revolutions have occurred in robotics, computer vision, and knowledge representation. A better understanding of the problems and their complexity properties, combined with increased mathematical sophistication, has led to workable research agendas and robust methods. Although increased formalization and specialization led fields such as vision and robotics to become somewhat isolated from “mainstream” AI in the 1990s, this trend has reversed in recent years as tools from machine learning in particular have proved effective for many problems. The process of reintegration is already yielding significant benefits

1.3.9 The emergence of intelligent agents (1995–present)

Perhaps encouraged by the progress in solving the subproblems of AI, researchers have also started to look at the “whole agent” problem again. The work of Allen Newell, John Laird, and Paul Rosenbloom on SOAR (Newell, 1990; Laird *et al.*, 1987) is the best-known example of a complete agent architecture. One of the most important environments for intelligent agents is the Internet. AI systems have become so common in Web-based applications that the “-bot” suffix has entered everyday language. Moreover, AI technologies underlie many

Internet tools, such as search engines, recommender systems, and Web site aggregators.

One consequence of trying to build complete agents is the realization that the previously isolated subfields of AI might need to be reorganized somewhat when their results are to be tied together. In particular, it is now widely appreciated that sensory systems (vision, sonar, speech recognition, etc.) cannot deliver perfectly reliable information about the environment. Hence, reasoning and planning systems must be able to handle uncertainty. A second major consequence of the agent perspective is that AI has been drawn into much closer contact with other fields, such as control theory and economics, that also deal with agents. Recent progress in the control of robotic cars has derived from a mixture of approaches ranging from better sensors, control-theoretic integration of sensing, localization and mapping, as well as a degree of high-level planning.

Despite these successes, some influential founders of AI, including John McCarthy (2007), Marvin Minsky (2007), Nils Nilsson (1995, 2005) and Patrick Winston (Beal and Winston, 2009), have expressed discontent with the progress of AI. They think that AI should put less emphasis on creating ever-improved versions of applications that are good at a specific task, such as driving a car, playing chess, or recognizing speech. Instead, they believe AI should return to its roots of striving for, in Simon's words, "machines that think, that learn and that create." They call the effort **human-level AI** or HLAI; their first symposium was in 2004 (Minsky *et al.*, 2004). The effort will require very large knowledge bases; Hendler *et al.* (1995) discuss where these knowledge bases might come from.

A related idea is the subfield of **Artificial General Intelligence** or AGI (Goertzel and Pennachin, 2007), which held its first conference and organized the *Journal of Artificial General Intelligence* in 2008. AGI looks for a universal algorithm for learning and acting in any environment, and has its roots in the work of Ray Solomonoff (1964), one of the attendees of the original 1956 Dartmouth conference. Guaranteeing that what we create is really **Friendly AI** is also a concern (Yudkowsky, 2008; Omohundro, 2008), one we will return to in Chapter 26.

1.3.10 The availability of very large data sets (2001–present)

Throughout the 60-year history of computer science, the emphasis has been on the *algorithm* as the main subject of study. But some recent work in AI suggests that for many problems, it makes more sense to worry about the *data* and be less picky about what algorithm to apply. This is true because of the increasing availability of very large data sources: for example, trillions of words of English and billions of images from the Web (Kilgariff and Grefenstette, 2006); or billions of base pairs of genomic sequences (Collins *et al.*, 2003).

One influential paper in this line was Yarowsky's (1995) work on word-sense disambiguation: given the use of the word "plant" in a sentence, does that refer to flora or factory? Previous approaches to the problem had relied on human-labeled examples combined with machine learning algorithms. Yarowsky showed that the task can be done, with accuracy above 96%, with no labeled examples at all. Instead, given a very large corpus of unannotated text and just the dictionary definitions of the two senses—"works, industrial plant" and "flora, plant life"—one can label examples in the corpus, and from there **bootstrap** to learn

HUMAN-LEVEL AI

ARTIFICIAL GENERAL
INTELLIGENCE

FRIENDLY AI

new patterns that help label new examples. Banko and Brill (2001) show that techniques like this perform even better as the amount of available text goes from a million words to a billion and that the increase in performance from using more data exceeds any difference in algorithm choice; a mediocre algorithm with 100 million words of unlabeled training data outperforms the best known algorithm with 1 million words.

As another example, Hays and Efros (2007) discuss the problem of filling in holes in a photograph. Suppose you use Photoshop to mask out an ex-friend from a group photo, but now you need to fill in the masked area with something that matches the background. Hays and Efros defined an algorithm that searches through a collection of photos to find something that will match. They found the performance of their algorithm was poor when they used a collection of only ten thousand photos, but crossed a threshold into excellent performance when they grew the collection to two million photos.

Work like this suggests that the “knowledge bottleneck” in AI—the problem of how to express all the knowledge that a system needs—may be solved in many applications by learning methods rather than hand-coded knowledge engineering, provided the learning algorithms have enough data to go on (Halevy *et al.*, 2009). Reporters have noticed the surge of new applications and have written that “AI Winter” may be yielding to a new Spring (Havenstein, 2005). As Kurzweil (2005) writes, “today, many thousands of AI applications are deeply embedded in the infrastructure of every industry.”

1.4 THE STATE OF THE ART

What can AI do today? A concise answer is difficult because there are so many activities in so many subfields. Here we sample a few applications; others appear throughout the book.

Robotic vehicles: A driverless robotic car named STANLEY sped through the rough terrain of the Mojave dessert at 22 mph, finishing the 132-mile course first to win the 2005 DARPA Grand Challenge. STANLEY is a Volkswagen Touareg outfitted with cameras, radar, and laser rangefinders to sense the environment and onboard software to command the steering, braking, and acceleration (Thrun, 2006). The following year CMU’s BOSS won the Urban Challenge, safely driving in traffic through the streets of a closed Air Force base, obeying traffic rules and avoiding pedestrians and other vehicles.

Speech recognition: A traveler calling United Airlines to book a flight can have the entire conversation guided by an automated speech recognition and dialog management system.

Autonomous planning and scheduling: A hundred million miles from Earth, NASA’s Remote Agent program became the first on-board autonomous planning program to control the scheduling of operations for a spacecraft (Jonsson *et al.*, 2000). REMOTE AGENT generated plans from high-level goals specified from the ground and monitored the execution of those plans—detecting, diagnosing, and recovering from problems as they occurred. Successor program MAPGEN (Al-Chang *et al.*, 2004) plans the daily operations for NASA’s Mars Exploration Rovers, and MEXAR2 (Cesta *et al.*, 2007) did mission planning—both logistics and science planning—for the European Space Agency’s Mars Express mission in 2008.

Game playing: IBM's DEEP BLUE became the first computer program to defeat the world champion in a chess match when it bested Garry Kasparov by a score of 3.5 to 2.5 in an exhibition match (Goodman and Keene, 1997). Kasparov said that he felt a "new kind of intelligence" across the board from him. *Newsweek* magazine described the match as "The brain's last stand." The value of IBM's stock increased by \$18 billion. Human champions studied Kasparov's loss and were able to draw a few matches in subsequent years, but the most recent human-computer matches have been won convincingly by the computer.

Spam fighting: Each day, learning algorithms classify over a billion messages as spam, saving the recipient from having to waste time deleting what, for many users, could comprise 80% or 90% of all messages, if not classified away by algorithms. Because the spammers are continually updating their tactics, it is difficult for a static programmed approach to keep up, and learning algorithms work best (Sahami *et al.*, 1998; Goodman and Heckerman, 2004).

Logistics planning: During the Persian Gulf crisis of 1991, U.S. forces deployed a Dynamic Analysis and Replanning Tool, DART (Cross and Walker, 1994), to do automated logistics planning and scheduling for transportation. This involved up to 50,000 vehicles, cargo, and people at a time, and had to account for starting points, destinations, routes, and conflict resolution among all parameters. The AI planning techniques generated in hours a plan that would have taken weeks with older methods. The Defense Advanced Research Project Agency (DARPA) stated that this single application more than paid back DARPA's 30-year investment in AI.

Robotics: The iRobot Corporation has sold over two million Roomba robotic vacuum cleaners for home use. The company also deploys the more rugged PackBot to Iraq and Afghanistan, where it is used to handle hazardous materials, clear explosives, and identify the location of snipers.

Machine Translation: A computer program automatically translates from Arabic to English, allowing an English speaker to see the headline "Ardogan Confirms That Turkey Would Not Accept Any Pressure, Urging Them to Recognize Cyprus." The program uses a statistical model built from examples of Arabic-to-English translations and from examples of English text totaling two trillion words (Brants *et al.*, 2007). None of the computer scientists on the team speak Arabic, but they do understand statistics and machine learning algorithms.

These are just a few examples of artificial intelligence systems that exist today. Not magic or science fiction—but rather science, engineering, and mathematics, to which this book provides an introduction.

1.5 SUMMARY

This chapter defines AI and establishes the cultural background against which it has developed. Some of the important points are as follows:

- Different people approach AI with different goals in mind. Two important questions to ask are: Are you concerned with thinking or behavior? Do you want to model humans or work from an ideal standard?